

**COMSATS University Islamabad,
Sahiwal Campus.**

RadiantAura

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2021-2025)

By

Ayesha Akram	CIIT/FA21-BCS-081/SWL
Moomna Nazir	CIIT/FA21-BCS-106/SWL
Muhammad Ibraheem	CIIT/FA21-BCS-108/SWL
Uqba	CIIT/FA21-BCS-143/SWL



**COMSATS University Islamabad,
Sahiwal Campus.**

RadiantAura

By

Ayesha Akram	CIIT/FA21-BCS-081/SWL
Moomna Nazir	CIIT/FA21-BCS-106/SWL
Muhammad Ibraheem	CIIT/FA21-BCS-108/SWL
Uqba	CIIT/FA21-BCS-143/SWL

Supervisor

Dr. Tariq Shahzad

Bachelor of Science in Computer Science (2021-2025)

The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of other.

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Ayesha Akram

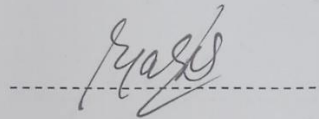
M. Ibraheem

Moomna Nazir

Uqba

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) "RadiantAura" was developed by Uqba (CIIT/FA21-BCS-143/SWL), Moomna Nazir (CIIT/FA21-BCS-106/SWL), Ayesha Akram (CIIT/FA21-BCS-081/SWL), and Muhammad Ibraheem (CIIT/FA21-BCS-108/SWL), under the supervision of "Dr. Tariq Shahzad" and that in his opinion; it is fully adequate, in scope and qualify for the degree of Bachelors of Science in Computer Science.



Supervisor
Dr. Tariq Shahzad
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus



External Examiner
Dr Iqra Muneer
Assistant Professor
Department of Computer Science,
UET Narowal Campus

Head of Department
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

RadiantAura is an innovative beauty consultancy platform designed to provide personalized skincare and makeup recommendations through AI-driven solutions. Unlike traditional beauty advisory systems, it offers a comprehensive, user-friendly experience by integrating advanced face analysis and machine learning algorithms to deliver tailored product suggestions. Users can securely register using email and password, ensuring the protection of sensitive data through robust authentication mechanisms.

The platform features an intuitive interface designed for effortless navigation, allowing users to scan their face, receive personalized recommendations, and interact with an AI-powered chatbot for real-time beauty assistance. It offers both, an Android application for end-users and a web-based admin panel for managing user data, product catalogs, and system monitoring.

Following Agile development methodologies, **RadiantAura** undergoes continuous improvements based on user feedback and evolving market needs. Functional requirements such as face analysis, user authentication, and recommendation generation drive the core features, while non-functional aspects like security, scalability, and usability ensure a seamless user experience.

The system undergoes rigorous testing, including unit testing, integration testing, and system testing, to validate its functionality and reliability. User acceptance testing is also conducted to refine usability and ensure the platform meets user expectations effectively.

In summary, **RadiantAura** stands as a pioneering beauty consultancy platform, offering personalized recommendations, secure user interactions, and advanced AI-driven insights. By providing a unified solution for beauty consultation, it empowers users to make informed decisions, enhancing their beauty routines with cutting-edge technology.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Dr. Tariq Shahzad**”. Without his supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to him for his encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Ayesha Akram

M. Ibraheem

Moomna Nazir

Uqba

Abbreviations

API	Application Programming Interface
JS	JavaScript
NLP	Natural Language Processing
IDEs	Integrated Development Environments
MVC	Model View Control
VS Code	Visual Studio Code
HTML	Hypertext Markup Language
CSS	Cascading Style Sheet
SDK	Software Development Kit
UML	Unified Modeling Language
UI/UX	User Interface / User Experience
CNN	Convolutional Neural Network
SEO	Search Engine Optimization

URLs	Universal Resource Locaters
SDLC	Software Development Life Cycle
NB Classifier	Naïve Bayes Classifier

Table of Contents

Chapter 1	
1 Introduction.....	1
1.1 Brief Overview.....	1
1.2 Relevance to Course Modules	3
1.2.1 Web Engineering.....	3
1.2.2 Introduction to Software Engineering.....	3
1.2.3 Database Design	4
1.2.4 Human-Computer Interaction	4
1.2.5 Software Design and Architecture	5
1.2.6 Object-Oriented Programming	5
1.2.7 Project Management	5
1.2.8 Artificial Intelligence (AI) Basics.....	6
1.2.9 Mobile Application Development.....	6
1.2.10 Computer Networks	7
1.3 Project Background.....	7
1.4 Literature Review	9
1.5 Analysis from Literature Review	10
1.5.1 Addressing the Lack of Direct Personalization and Control	11
1.5.2 Real-Time Personalized Beauty Advice	11
1.5.3 Moving Beyond Brand-Limited Responses.....	11
1.5.4 Cross-Platform Accessibility	11
1.5.5 Enhancing Skin Analysis with Product Suggestions	12
1.5.6 Administrative Efficiency and Backend Control.....	12

1.6	Scope	12
1.7	Methodology and Software Lifecycle	13
1.7.1	Design Methodology	13
1.7.2	Software Process Model.....	13
1.8	Rationale behind Selected Methodology	15
1.8.1	Model-View-Controller (MVC) architecture	15
1.8.2	Agile Development Process Model	16
Chapter 2		
2	Problem Definition.....	18
2.1	Problem Statement	18
2.2	Deliverables and Development Requirements	19
2.2.1	RadiantAura Platform.....	20
2.2.2	Documentation.....	20
2.2.3	Testing Reports	21
2.2.4	Deployment Setup.....	21
2.3	Development Requirements	22
2.3.1	Hardware Requirements.....	22
2.3.2	Software Requirements	23
2.3.3	Server and Hosting Requirements.....	24
Chapter 3		
3	Requirement Analysis.....	25
3.1	Use Case Descriptions	27
3.1.1	UC-01: Authentication	27
3.1.2	UC-02: Get Registration	27

3.1.3	UC-03: Choose options	29
3.1.4	UC-04: Chat.....	29
3.1.5	UC-05: Scan Face.....	30
3.2	Functional Requirements	31
3.2.1	User Registration and Authentication	31
3.2.2	Face Analysis Module	32
3.2.3	Personalized Recommendation Module	32
3.2.4	Cart Management	33
3.2.5	Chatbot.....	33
3.2.6	Admin Panel Management	33
3.2.7	Product Browsing and Categorization	34
3.3	Nonfunctional Requirements	34
3.3.1	Performance.....	34
3.3.2	Security.....	35
3.3.3	Scalability.....	35
3.3.4	Accessibility	35
3.3.5	Reliability.....	35
3.3.6	Usability	36
Chapter 4		
4	Design and Architecture.....	37
4.1	System Architecture.....	37
4.2	Data Representation	38
4.3	Process Flow/Representation	40
4.3.1	Activity Diagram	41

4.4	Sequence Diagram.....	44
4.4.1	Sequence Diagram (User Interaction).....	45
4.4.2	Sequence Diagram (Admin Interaction).....	46
4.5	Class Diagram	48
Chapter 5		
5	Implementation	50
5.1	Laravel Framework and the MVC Architecture	50
5.2	Development Tools and Technologies.....	51
5.2.1	Frontend.....	51
5.2.2	Backend	51
5.2.3	Database	51
5.2.4	Other Tools	52
5.3	User Registration Algorithm.....	52
5.4	Face Analysis and Recommendation Algorithm	52
5.5	Security and Data Protection.....	52
5.6	Admin Panel	52
5.6.1	Admin Capabilities	52
5.7	Application Integration and Optimization.....	53
5.7.1	SEO	53
5.7.2	Database Optimization	53
5.7.3	Security Measures	53
5.7.4	Responsive Design.....	53
5.8	Deployment Details.....	54
5.8.1	Hosting Environment.....	54

5.8.2	Backend Deployment	54
5.8.3	Database Configuration.....	54
5.8.4	Mail Setup.....	54
5.8.5	Android App Deployment	54
5.8.6	Version Control.....	55
5.9	User interface	55
5.9.1	Welcome Screen	55
5.9.2	RadiantAura Features	56
5.9.3	Sign Up Page.....	60
5.9.4	Login Page	61
5.9.5	Home Screen	62
5.9.6	Product Categories	63
5.9.7	Admin Panel	64
Chapter 6		
6	Testing.....	66
6.1	Unit Testing	66
6.2	Manual Testing	67
6.3	Functional Testing.....	68
6.4	Integration Testing	70
6.5	System Testing.....	71
Chapter 7		
7	Conclusion and Future Work	73
7.1	Conclusion	73
7.2	Future Work.....	74

7.2.1	Augmented Reality (AR) Integration	74
7.2.2	Advanced AI-Based Personalized Recommendations.....	74
7.2.3	Community Reviews and Product Ratings	75
7.2.4	Subscription Plans and Loyalty Programs	75
7.2.5	Globalization Support (Languages and Currency)	75
7.2.6	Social Media and Influencer Integration	75
7.2.7	Advanced Admin Dashboard and Analytics.....	75
7.2.8	Accessibility Improvements.....	76
Chapter 8		
References		77

List of Figures

Figure 1.1: Agile Model.....	15
Figure 3.1: Use Case	26
Figure 4.1: System Architecture	37
Figure 4.2: DFD for User.....	39
Figure 4.3: DFD for Admin	40
Figure 4.4: Process Flow Diagram.....	43
Figure 4.5: Activity Diagram for Admin	44
Figure 4.6: Sequence Diagram for User	46
Figure 4.7: Sequence Diagram for Admin	48
Figure 4.8: Class Diagram	49
Figure 5.1: App Welcome Screen.....	56
Figure 5.2: App HomePage.....	57
Figure 5.3: App Chatbot	58
Figure 5.4: Web Face Scanner	59
Figure 5.5: Face Scanner in App.....	60
Figure 5.6: SignUp Page	61
Figure 5.7: Login Page.....	62
Figure 5.8: RadiantAura Website Homepage	63
Figure 5.9: Product Categories	64
Figure 5.10: Admin Panel	64

List of Tables

Table 1.1: Comparison with other similar Systems.....	9
Table 3.1: Use Case Authentication.....	27
Table 3.2: Use case Get Registration.....	28
Table 3.3: Use Case Choose options	29
Table 3.4: Use case Chat.....	30
Table 6.1: Unit Testing.....	67
Table 6.2: Manual Testing.....	67
Table 6.3: Functional Testing	70
Table 6.4: Integration testing	71
Table 6.5: System Testing.....	72

Chapter 1

1 Introduction

With the hyper-connected world of today, the experience of consumers interacting with beauty and skincare products has changed significantly. Trial-and-error shopping or receiving generic recommendations while buying makeup or skincare products are no longer the case. The contemporary beauty lover expects individualized, data-led experiences that are accessible, quick, and precise. Sensing the change, RadiantAura was envisioned and created as a cutting-edge AI-driven skincare and makeup platform, a website and mobile application.

The motivation for RadiantAura is the increasing demand for personalized solutions to beauty, particularly in places such as Pakistan, where skincare suggestions tend to be generic, uninformed, or rooted in ancient, non-scientific traditions. Most consumers are left confused when picking the appropriate products suited to their individual skin types, depending on arbitrary advice online or word of mouth. This leads to inappropriate product choices, money down the drain, and unsuccessful skincare regimens.

RadiantAura remedies these issues by providing a clever, intuitive digital world where users can access expert-level advice seamlessly. The site features an AI-enhanced chatbot that gives skincare and makeup recommendations based on the personal needs of every user. Users also have the ability to use a face scanner, which scans the user's skin type in real time and offers suggestions on the right products and practices based on AI analysis.

Not only does this smart system enhance user experience via accuracy and customization, it also enables users to make smart decisions without having to refer to dermatologists or beauty experts in person. From choosing a moisturizer for oily skin to picking just the right shade of foundation, RadiantAura fuses beauty and technology to provide a smooth, science-based skincare experience for any user.

1.1 Brief Overview

RadiantAura is a cutting-edge beauty consultancy platform developed to transform the way individuals engage with skincare and makeup in the digital era. As consumer expectations evolve and the demand for customized beauty solutions continues to grow, RadiantAura offers an intelligent, user-centric platform that empowers users to better understand their skin, receive tailored product recommendations, and make confident beauty purchases all from the comfort of their home.

At its core, RadiantAura leverages advanced machine learning algorithms and a responsive chatbot system to deliver highly personalized beauty experiences. Users can analyze their skin type and condition using an intuitive interface, allowing the platform to generate customized skincare routines and makeup suggestions that align with their specific needs and goals. This approach not only removes the guesswork traditionally involved in product selection but also ensures that users receive guidance grounded in real-time data and scientific analysis.

One of RadiantAura's most innovative features is its AI-powered facial analysis module, which enables users to upload selfies for a comprehensive skin assessment. This feature utilizes computer vision and machine learning to evaluate key skin characteristics such as tone, texture, dryness, and acne-prone areas offering instant feedback and personalized advice. By incorporating this technology, RadiantAura helps users make informed decisions about their beauty regimens, enhancing both the effectiveness and satisfaction of their skincare routines.

The platform's intelligent chatbot further enriches the user experience by providing round-the-clock access to virtual beauty consultations. Whether users are seeking advice on product usage, tips for maintaining healthy skin, or assistance navigating the platform, the chatbot offers real-time, context-aware support that mimics the experience of speaking with a professional consultant. This conversational interface bridges the gap between technology and human expertise, delivering guidance that is both accessible and reliable.

RadiantAura also places a strong emphasis on cross-platform accessibility and user convenience. The system is available as both a responsive web application and a dedicated Android mobile app, allowing users to interact with the platform seamlessly across different devices. This multi-platform functionality ensures that users can receive consistent support and product access whether they are at home, at work, or on the go.

From a technical standpoint, RadiantAura is built with scalability, performance, and security in mind. The platform utilizes **Android Studio** for mobile application development and **Visual Studio** for the web-based frontend and backend systems. It is developed using a robust tech stack that includes **Kotlin**, **MySQL**, and **PHP**, ensuring reliability and ease of integration with future technologies. Additionally, user data privacy and system security are top priorities. RadiantAura incorporates secure authentication protocols, encrypted data storage, and access control measures to protect sensitive user information from unauthorized access.

What truly sets RadiantAura apart is its commitment to combining technological innovation with practical beauty expertise. By seamlessly merging AI-driven analysis, personalized recommendations, and an engaging user interface, the platform redefines what it means to

receive beauty advice in the digital age. RadiantAura not only simplifies product discovery and selection but also empowers users to make smarter, faster, and more informed decisions about their skincare and makeup needs.

In a world where personalization and convenience are paramount, RadiantAura delivers a next-generation beauty consultancy experience intelligent, interactive, and entirely tailored to you.

1.2 Relevance to Course Modules

The development of RadiantAura draws upon a wide range of core concepts, techniques, and methodologies learned throughout our academic journey. These foundational courses provided both the technical depth and user-focused insights required to conceptualize, design, and implement a full-scale web and mobile applications. Below is a breakdown of how each relevant course contributed to shaping various aspects of the project:

1.2.1 Web Engineering

This course equipped us with the foundational and advanced skills required to develop dynamic, secure, and user-friendly web applications. The practical knowledge of HTML, CSS, JavaScript, and Bootstrap was essential for designing responsive frontend interfaces for candidates, employers, and administrators. Furthermore, the project implementation in Laravel, a powerful PHP-based web framework, allowed us to apply key concepts such as Model View Control (MVC) architecture, routing, middleware, session management, and form validation in a structured and efficient manner. Laravel's built-in features significantly enhanced the development process by streamlining database interactions, authentication, and client-server communication. Overall, the course enabled the creation of a full-featured, modular platform that ensures a smooth and secure user experience across all components.[1]

1.2.2 Introduction to Software Engineering

The structured methodologies taught in this course played a key role in shaping our project management and planning approach for the development of the RadiantAura beauty consultancy platform. We adopted Software Development Life Cycle (SDLC) models to guide us through well-defined phases, including requirement gathering, system design, implementation, testing, and deployment[2]. This systematic approach enabled us to align the platform's features with the actual needs of its user's skincare and makeup enthusiasts seeking personalized beauty solutions. By carefully planning and managing each stage, we ensured that

RadiantAura addressed core user expectations such as AI-powered skin analysis, tailored product recommendations, seamless cross-platform access, and real-time chatbot support, ultimately delivering a user-focused and technically robust solution.

1.2.3 Database Design

A solid understanding of database management was essential to the successful development of the RadiantAura. The knowledge gained from this course directly influenced the design and implementation of a well-structured and efficient database schema. We applied relational database principles to organize and manage data related to users, product listings, skin analysis results, chatbot interactions, and authentication securely and efficiently. Using MySQL, we ensured data consistency, integrity, and fast retrieval across the platform's various modules[3]. This foundational understanding enabled us to support real-time interactions, personalized recommendations, and seamless cross-platform functionality, all while maintaining scalability and performance.

1.2.4 Human-Computer Interaction

The principles of Human-Computer Interaction (HCI) played a vital role in shaping the overall design and user experience of the RadiantAura platform, across both the web application and Android mobile app. Our focus was on creating an intuitive, accessible, and visually engaging interface that meets the diverse needs of users, including beauty enthusiasts, skincare shoppers, and first-time visitors. Guided by HCI best practices, we ensured that the user interface was consistent and responsive across different devices, maintaining smooth navigation and clear interaction pathways.

Key usability features such as a well-organized dashboard, smart product listings, AI-driven skin analysis, seamless chatbot interaction, and advanced search functionality were carefully designed to minimize cognitive load and support task efficiency. Special attention was given to accessibility, ensuring that the platform remained inclusive and user-friendly for a wide audience. By integrating these principles into both the mobile and web versions, we delivered a cohesive and satisfying user experience that enhances engagement and supports user goals effectively.

1.2.5 Software Design and Architecture

This course played a crucial role in guiding the architectural decisions of RadiantAura. We implemented the MVC pattern using Laravel to separate business logic, data access, and presentation layers. This ensured modularity, maintainability, and scalability. The use of service layers, controllers, middleware, and Blade templating enabled us to construct a system that could easily accommodate future features like AI-skin tone detection.

1.2.6 Object-Oriented Programming

While Laravel inherently supports object-oriented principles, our implementation emphasized practical application over deep abstraction. We applied object oriented programming (OOP) concepts such as encapsulation and modular design to structure user roles, controllers, and models efficiently. The use of Laravel's resource controllers, route model binding, and Eloquent ORM allowed us to manage data consistently without overcomplicating the architecture. This pragmatic approach facilitated faster development cycles, easier debugging, and greater flexibility in feature expansion [4].

Together, these course modules provided the interdisciplinary knowledge needed to transform RadiantAura from an idea into a fully functional, AI-driven skin advisor that serves the real-world demands of modern beauty ecosystems.

1.2.7 Project Management

This subject offered valuable insights into the structured planning, execution, monitoring, and control of software projects. The principles and methodologies learned were directly applied throughout the development of the RadiantAura beauty consultancy platform. From defining clear project objectives and timelines to allocating resources and managing risks, project management practices ensured a disciplined and organized workflow.

We used tools and techniques such as Gantt charts, task breakdown structures, and progress tracking to manage development phases effectively covering requirement analysis, design, implementation, testing, and deployment. These practices were crucial in coordinating tasks across both the web and Android app development, maintaining clear communication within the team, and ensuring that deliverables met quality standards within the set deadlines. Overall, project management principles played a central role in delivering RadiantAura as a functional, user-focused, and timely digital solution.

1.2.8 Artificial Intelligence (AI) Basics

The development of the chatbot in RadiantAura platform incorporated several foundational and practical AI techniques, reflecting the core principles learned in this subject. AI was applied in multiple modules to enhance user experience through intelligent automation and personalization.

We trained a GPT-2 model to generate personalized skincare advice based on user queries, allowing the chatbot to provide more natural, human-like responses. For facial analysis, we implemented a Convolutional Neural Network (CNN) to scan uploaded selfies and identify skin types, enabling the system to deliver tailored skincare product recommendations. Additionally, a Naive Bayes (NB) classifier was used to interpret user input text and categorize it for basic chatbot interactions, providing context-aware responses where deep learning wasn't necessary.

These AI-driven components demonstrate a clear understanding of both traditional and modern AI methods. By combining rule-based logic, machine learning, and deep learning approaches, we successfully enhanced the platform's intelligence, making user interactions more meaningful and effective.

1.2.9 Mobile Application Development

This subject provided a comprehensive foundation for building robust and user-friendly mobile applications. The skills and concepts learned were directly applied in the development of the RadiantAura Android application using Kotlin, a modern and efficient programming language designed for Android development.

The app was built to offer a seamless and intuitive user experience, allowing users to interact with features such as AI-powered skincare analysis, a smart chatbot for product recommendations, and a dynamic product catalog. Core Android components including activities, fragments, recycler views, and view models were utilized to create a responsive and modular architecture. Retrofit was integrated to handle communication with backend services, including a Flask API for skin analysis and product recommendations.

Special attention was given to user interface and user experience (UI/UX) design to ensure consistency and accessibility across different devices, while asynchronous tasks and lifecycle-aware components helped maintain performance and reliability. This hands-on experience with Kotlin and Android Studio allowed us to transform theoretical knowledge into a fully functional mobile solution, bridging the gap between user needs and technology.

1.2.10 Computer Networks

The Radiant Aura likely communicates with a backend server or a cloud database via HTTP requests. This involves understanding of client-server architecture, protocols like HTTP/HTTPS, REST APIs, and even WebSocket for real-time updates. Knowledge from networking courses enabled smooth data transmission and ensured secure, fast communication.

1.3 Project Background

The beauty and skincare industry has seen a significant transformation in recent years due to the rapid advancement of digital technologies and artificial intelligence. As consumers become more informed and technology-savvy, their expectations for personalization, convenience, and efficiency in beauty-related services have increased substantially. This shift has opened the door for innovative platforms that can deliver personalized experiences beyond traditional in-store consultations or generic e-commerce product listings. However, despite the growing presence of online beauty services, many users still encounter critical limitations that hinder their ability to find the right products or guidance tailored to their unique skin types and beauty goals.

One of the most common challenges user's faces is the lack of personalized skincare and makeup recommendations. Most online platforms rely on static filters and general product categories, which do not take into account a user's specific skin tone, skin condition, or preferences. As a result, users often spend excessive time browsing through irrelevant products or experimenting with trial-and-error purchases. Another issue is the absence of accurate skin diagnosis tools that can help users understand their skin concerns before choosing products. Additionally, beauty advice offered online is often inconsistent, subjective, and not based on scientific analysis, making it unreliable for those who seek genuine skincare improvement.

To address these issues, the RadiantAura project was conceptualized as a complete beauty consultancy platform that leverages artificial intelligence and modern web and mobile development technologies to offer smart, accurate, and personalized beauty solutions. RadiantAura aims to empower users by offering features that combine machine learning, image analysis, natural language processing (NLP), and user-centered design, all within a seamless and interactive platform.

At the heart of RadiantAura lies its AI-powered face analysis module, which allows users to upload selfies and receive a real-time skin assessment. This module was developed using CNNs to scan facial features and detect skin types (e.g., oily, dry, combination) and conditions such as acne, dryness, or pigmentation. The analysis results are then used to recommend suitable skincare and makeup products specifically matched to the user's profile, improving accuracy and relevance in product suggestions.

In addition to image-based analysis, it also features an intelligent chatbot system that offers personalized skincare advice and product guidance. This chatbot uses a combination of Naive Bayes classification to categorize text inputs and a custom-trained GPT-2 model to generate human-like, context-aware responses. This hybrid approach allows the chatbot to not only answer user queries efficiently but also simulate real beauty consultation sessions, improving user engagement and satisfaction.

To ensure a broad reach and user accessibility, RadiantAura was developed as both a mobile application and a web platform. The Android app was built using Kotlin in Android Studio, allowing users to engage with the platform on the go. It includes features such as image uploading, real-time chatbot communication, and personalized product browsing. The web version was built using PHP, HTML, CSS, and JavaScript, providing users with access through desktop and tablet devices. Both versions are connected to a central backend, which includes a Flask API for AI functionalities and a MySQL database for managing user data, product information, and chatbot responses.

Retrofit was integrated into the Android app to enable secure and smooth communication between the mobile interface and the backend services. The platform also includes an admin panel that allows administrators to manage product inventories, monitor user activity, and

update chatbot knowledge all with proper user authentication and role-based access controls to ensure privacy and data protection.

RadiantAura’s development followed the Agile methodology, promoting incremental development and continuous integration. Through regular feedback loops, the platform evolved to meet actual user needs while maintaining a focus on quality, usability, and performance. The system is also built with scalability in mind, allowing future integration of more advanced AI models, user features such as reviews and feedback, and expanded product lines.

Ultimately, RadiantAura redefines the concept of digital beauty consultancy by integrating cutting-edge technologies with practical user needs. It offers a complete, AI-enhanced experience that is not only technically sound but also emotionally resonant helping users feel more confident and informed about their beauty choices. This platform demonstrates how the thoughtful application of AI, mobile development, and human-computer interaction principles can bring about meaningful improvements in one of the most personal and rapidly growing industries today.

1.4 Literature Review

The development of RadiantAura was influenced by a review of skin care products and consultations platforms, both local and international. While these platforms offer useful functionalities, most either lack real-time face scanning for skin type detection or are not tailored to the specific needs of Pakistanis. Below is a comparative analysis of platforms that relates to the concept of RadiantAura and how our system builds upon or differs from them.

Table 1.1: Literature Review

ID	Application Name	Weakness	Proposed Project Solution
[5]	BeautyAdvisor	Limited control over beauty professionals and service quality due to intermediary role.	The proposed project for the makeup app will overcome this limitation by providing users with direct access to personalized product recommendations and

			beauty tips, eliminating the need for intermediaries and ensuring a seamless and empowering beauty shopping experience.
[6]	BeautyBuddy	Difficulty in providing real-time beauty advice and personalized recommendations based on individual preferences.	Our project will leverage machine learning and user profiling to offer immediate, personalized beauty advice based on skin type and user can also use face scanner to detect their skin type and get personalized recommendations.
[7]	Dermalogica	Generates responses that only suggest to use their available skin care treatments.	Our system provides general but efficient skin care advices along with product recommendations.
[8]	Perfect Corp	It is available for iphones and ipads	Our project is a cross platform
[9]	Cetaphil	It tells the skin score like redness, oiliness, dark spots etc by scanning face and don't suggest any products or skin care advice	Our project not only scans the skin texture but also detects skin type and provides relevant advices

1.5 Analysis from Literature Review

The comprehensive review of existing skincare and beauty platforms has revealed multiple gaps that limit user experience and overall functionality. Applications such as BeautyAdvisor, BeautyBuddy, Dermalogica, Perfect Corp, and Cetaphil, while popular, exhibit several shortcomings in areas such as personalization, platform accessibility, and practical user

guidance. These insights have been instrumental in shaping the design, functionality, and innovation embedded within the RadiantAura platform.

1.5.1 Addressing the Lack of Direct Personalization and Control

One of the key limitations identified in platforms like BeautyAdvisor is their reliance on third-party professionals, which reduces control over the quality and consistency of the beauty recommendations[5]. RadiantAura overcomes this challenge by eliminating intermediaries and empowering users with direct access to personalized product recommendations and AI-driven beauty advice. This self-service approach ensures a more reliable, efficient, and empowering user experience. The system's chatbot and face analysis module work together to provide accurate, individualized guidance based on skin type and user preferences—something many traditional apps lack.

1.5.2 Real-Time Personalized Beauty Advice

Apps such as BeautyBuddy struggle to provide real-time, interactive support to users. In contrast, RadiantAura integrates machine learning models including GPT-2 for advice generation and CNN for skin type detection to deliver real-time, personalized beauty consultations[6]. The interactive chatbot acts as a virtual beauty assistant, helping users make informed choices instantly. This high level of interaction increases user satisfaction and enhances platform credibility by making the beauty recommendation process feel more natural and user-specific.

1.5.3 Moving Beyond Brand-Limited Responses

Unlike Dermalogica, which restricts advice to its own brand of skincare treatments, RadiantAura adopts an open approach. The system offers generalized, effective skincare recommendations and a variety of product suggestions, allowing users to explore different brands and products that best suit their individual needs[7]. This broader spectrum enhances the platform's reliability and ensures users are not limited by a single product ecosystem.

1.5.4 Cross-Platform Accessibility

A significant limitation in Perfect Corp is its availability solely on iOS devices, which limits access for Android users[10]. RadiantAura addresses this by offering a cross-platform

experience, available as both a web application and an Android app developed in Kotlin. This inclusive design maximizes user reach and ensures accessibility regardless of the user's preferred device.

1.5.5 Enhancing Skin Analysis with Product Suggestions

While Cetaphil provides a face scanner that detects features like oiliness, redness, and dark spots, it does not offer actionable suggestions or product links based on the analysis. RadiantAura advances this concept by scanning facial features using CNN to determine the skin type and offering intelligent advice and personalized product recommendations. This end-to-end process from diagnosis to guidance transforms the user journey into an informative and actionable experience[9].

1.5.6 Administrative Efficiency and Backend Control

Most beauty platforms lack strong backend tools for product management and order tracking. RadiantAura includes a robust Admin Panel, enabling administrators to add, update, and delete products, manage orders, and track user activities with ease. This ensures efficient content and product management, making it suitable for both growing startups and established businesses.

1.6 Scope

The makeup app aspires to revolutionize the beauty industry by offering a comprehensive platform for users to discover, explore, and purchase makeup and skincare products. Its scope encompasses several key features aimed at enhancing the user experience. Through advanced machine learning algorithms, the app provides personalized product recommendations tailored to individual preferences, skin type, and beauty goals. Users can enjoy a seamless shopping experience, browsing through a diverse selection of products and making purchases directly within the app with a streamlined checkout process. Additionally, the app offers expert beauty advice through tutorials, tips, and insights to help users refine their beauty skills and confidence in makeup application and skincare routines. Prioritizing user privacy and security, the app ensures secure transactions for all purchases. With intuitive interfaces and easy navigation, the app simplifies the process of discovering new beauty products and trends, making beauty exploration effortless and enjoyable. Furthermore, users benefit from real-time customer

support and assistance via the app's chatbot feature, providing prompt guidance and solutions to any beauty-related queries or concerns.

1.7 Methodology and Software Lifecycle

To ensure the successful delivery of the RadiantAura platform, a well-defined software development methodology was adopted, balancing robust architectural planning with agile execution. The goal was to build a secure, scalable, and interactive skincare recommendation system powered by Laravel, machine learning, and real-time user interaction features. This section outlines the design methodology and process model that guided the development of RadiantAura from planning to deployment.

1.7.1 Design Methodology

For the RadiantAura, the development approach was centered around the Laravel framework, leveraging its built-in MVC architecture to maintain clean code organization and modular structure. While Laravel fully supports Object Oriented Programming (OOP) concepts such as inheritance, polymorphism, and encapsulation, our focus remained on a pragmatic, convention-driven approach that prioritized functionality, clarity, and modularity over deep object hierarchies. Core modules, including user authentication, chat and face scanning, product management, add to cart, and checkout, were developed using Laravel's native tools such as controllers, middleware, route grouping, service providers, and Blade templating engine. This ensured each component of the platform could be built, maintained, and tested independently, allowing for parallel development and smoother integration across teams. The design strategy emphasized scalability and maintainability. Each role-based feature (user, admin) was compartmentalized into isolated sections, allowing us to update or expand modules without introducing code-level conflicts. Laravel's Eloquent ORM was employed for seamless database interaction, while Blade templates were used to deliver dynamic content across all user interfaces. This lightweight, modular design methodology not only accelerated the development process but also enhanced system responsiveness, code reusability, and long-term maintainability.

1.7.2 Software Process Model

The development of RadiantAura followed the Agile Development Process Model, which allowed for iterative progress, continuous feedback, and incremental delivery of functional modules. Agile was selected due to the dynamic nature of the project, the evolving scope of

user requirements, and the necessity for active collaboration among team members. The project was divided into time-boxed sprints, each focused on delivering a specific feature or set of functionalities, such as the user registration flow, real time messaging system, face scanning, or admin analytics dashboard. At the end of each sprint, a working version of the platform was presented, tested, and reviewed. Feedback was incorporated immediately into the next sprint planning to ensure the platform evolved in alignment with user expectations and technical objectives[2]. Key elements of Agile implementation in RadiantAura include:

Sprint Planning

Defining priorities and allocating tasks across frontend, backend, and database teams.

Daily Stand-Ups

Short progress meetings to resolve blockers and align goals.

Continuous Integration & Testing

Regular deployment to local or staging servers for early validation.

Frequent Feedback Loops

Real-time feedback from peers and potential users helped refine usability, performance, and feature design.

This iterative, flexible approach ensured that the platform remained adaptive to changes, especially in response to UI/UX improvements or emerging technical challenges. It also reduced development risk by allowing early error detection and enabled the team to maintain steady progress without compromising quality. In summary, the combination of Laravel's modular design philosophy and Agile's responsive development cycle provided the ideal framework for delivering RadiantAura, a secure, scalable, and user-friendly solution built to meet the real-world demands of modern recruitment.

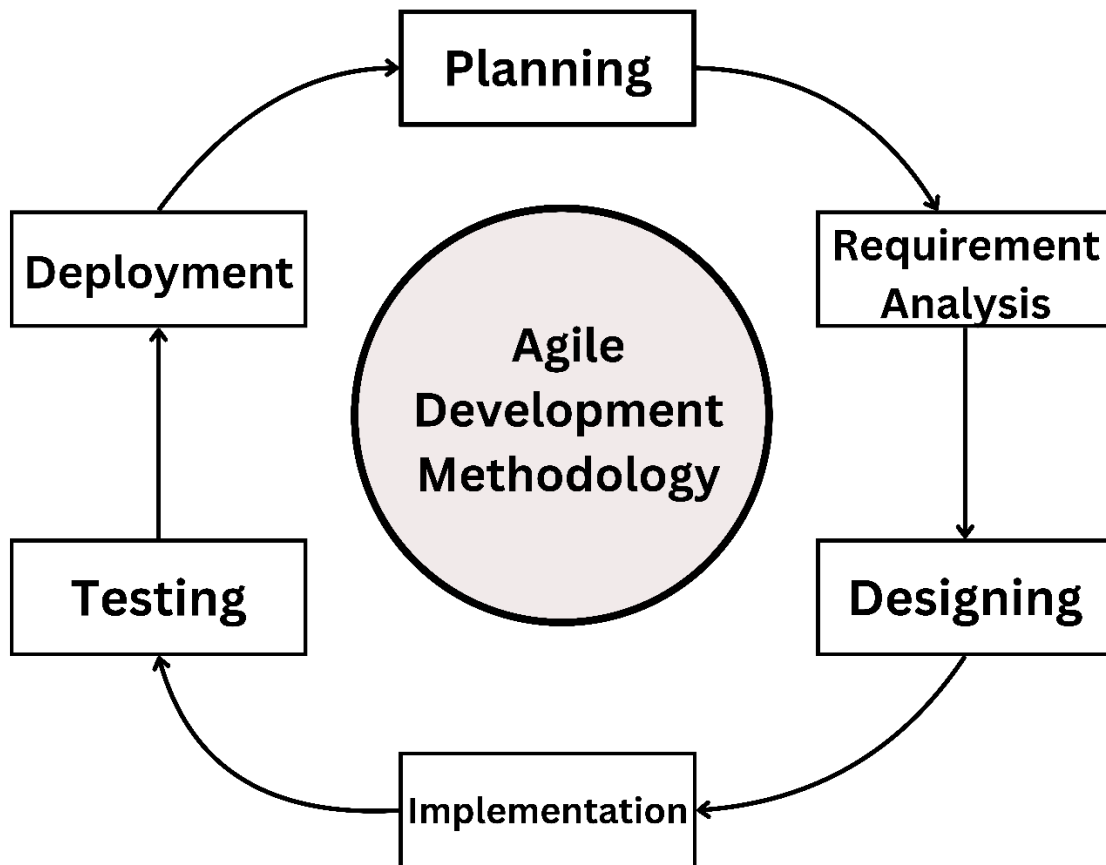


Figure 1.1: Agile Model

1.8 Rationale behind Selected Methodology

The selection of the Model-View-Controller (MVC) design architecture and the Agile Development Process Model for the development of RadiantAura was rooted in the platform's objectives, modularity, scalability, user-focused design, and adaptability to changing requirements. These methodologies provided a structured yet flexible foundation that aligned perfectly with the technical and functional needs of a AI assistant catering to multiple user roles: user and administrators.

1.8.1 Model-View-Controller (MVC) architecture

The MVC design pattern was adopted to ensure a clean separation of concerns across the system. In a platform like RadiantAura, where different user types perform distinct tasks, such as product browsing, chat, and face scanning, it was essential to create an architecture that could support clear role separation, parallel development, and ease of maintenance.

- **Model**

Represents the data layer of the system, managing user information, job postings, application records, messages, and activity logs via Laravel's Eloquent ORM.

- **View**

Handles the presentation layer using Laravel's Blade templating engine to deliver dynamic and responsive user interfaces for candidates, employers, and administrators.

- **Controller**

Acts as the intermediary, processing user input, managing logic, and orchestrating data flow between the model and the view.

This design not only supports scalability and reusability but also reduces interdependence, allowing modules to be developed, updated, or debugged independently. The MVC structure also ensures that future enhancements, such as AI driven skin tone suggestions, can be seamlessly incorporated without architectural disruption.

1.8.2 Agile Development Process Model

The Agile methodology was chosen due to its iterative, feedback-driven nature, which suits dynamic, user-centric applications like RadiantAura. Unlike traditional linear models, Agile promotes flexibility, continuous testing, and ongoing user engagement throughout the development cycle. Given the complexity and multi-role nature of the platform, Agile enabled the team to divide the project into focused sprints and prioritize features that deliver the most value to users in the shortest possible time[2]. Agile Implementation in RadiantAura:

- **Sprint Planning**

The project was divided into multiple sprints, each focusing on core modules such as user profile management, managing catalog, admin analytics dashboard, face scanning and messaging functionality. Each sprint was defined through user stories and feature priorities.

- **Incremental Delivery**

At the end of every sprint, a working feature or module was delivered and deployed to a staging environment. For instance, the admin dashboard and chatbot Module were completed and tested in early sprints before additional features like face scanning was layered in subsequent iterations.

- **Continuous Feedback Loop**

Regular feedback was obtained from potential users, and admins through surveys, walkthroughs, and demos. This feedback guided UI refinements, dashboard structuring, and workflow improvements.

- **Daily Stand-ups and Team Syncs**

Short, daily meetings helped team members track progress, address challenges, and realign priorities. This ensured cross-functional collaboration among developers, designers, and testers.

- **Adaptive Iterations**

Feature backlogs and sprint priorities were revised based on usability testing and user responses.

- **User-Centric Development**

Agile's focus on delivering value allowed us to tailor features like face scanning, chat consultations, and getting recommendations. Each feature was refined through actual user input and tested for performance and reliability.

Chapter 2

2 Problem Definition

Many people struggle to choose the right skincare or makeup products for their skin. They might buy a product that causes breakouts, or a foundation that doesn't match their tone or skin type. Walking into a store or browsing online can be overwhelming with so many options and so little guidance. Not everyone knows their skin type or even what ingredients to look for or avoid. This section outlines the problem statement and proposed solution.

2.1 Problem Statement

Navigating the world of skincare and beauty products is often an overwhelming and confusing journey for many consumers. The cosmetic market is flooded with thousands of options across a broad spectrum of brands, ingredients, skin types, and purposes. While this diversity offers choices, it also leads to significant challenges for users especially those who lack deep knowledge of skincare or struggle with specific skin concerns. The sheer volume of available products, coupled with conflicting online reviews and marketing claims, often leaves users uncertain about which items are genuinely suitable for their unique needs.

Many existing e-commerce and beauty platforms focus heavily on showcasing a wide range of products but fall short in offering the type of personalized guidance that modern consumers demand. These platforms typically rely on static product descriptions and generic recommendations, ignoring critical individual factors such as skin type (oily, dry, combination, sensitive), skin tone, allergies, and specific concerns like acne, aging, pigmentation, or dehydration. As a result, users frequently purchase products that are incompatible with their skin, leading to ineffective results, wasted money, and in some cases, adverse skin reactions.

Moreover, the traditional shopping experience lacks intelligent assistance and does not provide real-time consultation. Users often miss the expert guidance that could help them make informed decisions, especially when shopping online without the support of a dermatologist or beauty advisor. The absence of interactive tools such as facial scanning, AI-powered skin analysis, or dynamic chat support further contributes to a fragmented and frustrating user journey. In addition, users may find it tedious to compare products across categories, track their previous purchases, or receive updates aligned with their beauty goals.

Another challenge lies in the gap between product discovery and actual usage. Users often do not receive recommendations that align with their lifestyle preferences, such as vegan or cruelty-free options, specific ingredient preferences (e.g., niacinamide, salicylic acid), or formulations tailored for specific climates. Furthermore, many platforms neglect the post-purchase experience, offering limited support in terms of follow-up care, skin progress tracking, or ongoing guidance.

RadiantAura is designed to address these multifaceted challenges through an intelligent and user-centric approach. By integrating advanced machine learning algorithms, a skin-type prediction system using image analysis, and a conversational chatbot, RadiantAura delivers deeply personalized product recommendations. The platform considers individual characteristics, preferences, and skincare goals to guide users toward products that are most likely to benefit them. Through this personalized pathway, users can avoid trial-and-error routines and discover items that match their skin profile more precisely.

RadiantAura also simplifies the beauty shopping experience by offering real-time chatbot support, intelligent filtering, and an easy-to-navigate interface. Whether a user is searching for makeup that suits their undertones or skincare that addresses specific dermatological issues, the platform adapts to their needs dynamically. By bridging the gap between technological innovation and skincare personalization, RadiantAura empowers users to make confident, informed decisions transforming skincare from a confusing chore into a customized and enjoyable journey. Ultimately, the application seeks to revolutionize how users interact with beauty products, fostering trust, convenience, and efficacy in every step of their skincare and beauty routine.

2.2 Deliverables and Development Requirements

The successful implementation of RadiantAura involves the delivery of several key components, each contributing to the overall functionality and user experience of the platform. This section outlines the core deliverables that form the foundation of the project, including the mobile application, web-based platform, and AI-powered chatbot system. It also covers the essential documentation for users, developers, and administrators, as well as detailed testing reports to ensure software reliability and performance. Additionally, the deployment setup is

described to facilitate smooth release and maintenance across different environments. These deliverables collectively ensure that RadiantAura offers a robust, scalable, and personalized skincare and cosmetics solution tailored to user needs.

2.2.1 RadiantAura Platform

RadiantAura is a multi-platform skincare and cosmetics recommendation system designed to provide a personalized shopping and consultation experience. The mobile application is developed using Kotlin for Android, while the web application is built using Laravel (backend) and a responsive frontend with HTML, CSS, and JavaScript. The system integrates a smart chatbot powered by a Flask API and machine learning models trained to predict skin type and provide product recommendations based on user inputs, including text and captured facial images.

The mobile app allows users to register, log in securely, browse categorized beauty products (Skincare, Face, Lips), receive tailored recommendations, manage their cart, and complete purchases via integrated payment gateways. Similarly, the web platform offers a seamless browsing and shopping experience optimized for desktops and tablets. The chatbot is embedded in both platforms to assist users with real-time skincare advice, product suggestions, and guidance based on AI analysis.

The admin dashboard accessible via the web enables administrators to manage product listings, handle user queries, review chatbot interactions, process orders, and monitor analytics. This provides a centralized control system to manage all user-facing and backend operations efficiently.

2.2.2 Documentation

User Documentation

RadiantAura includes both in-app guidance and downloadable manuals for users. These resources explain how to register an account, upload or capture images for skin analysis, interact with the AI-powered chatbot, search and filter products, manage the shopping cart, complete purchases, and review order history.

Technical Documentation

This covers the full system architecture, including the Android app structure (activities, fragments, Retrofit integration), the Laravel backend (routes, controllers, models), and the Python-based AI engine (Flask API endpoints, skin prediction model details). It also includes

API documentation, database schema, integration flows, and deployment architecture for web and mobile components.

Admin Guide

A user-friendly manual for administrators to manage the backend via the web dashboard. It includes procedures for adding/editing/deleting products, updating order statuses (pending, processing, shipped, delivered), viewing sales reports, and moderating chatbot interactions to improve user experience.

2.2.3 Testing Reports

Testing of RadiantAura is conducted across all platforms to ensure a robust, bug-free user experience. Reports include:

- **Unit Testing:** For individual modules like user login, cart functions, and API responses.
- **Integration Testing:** Ensuring smooth communication between the Android app, web platform, and Flask-based chatbot.
- **UI/UX Testing:** Validating responsive design, layout rendering across different devices, and user navigation flows.
- **User Acceptance Testing (UAT):** Conducted on real devices (Android, desktops, tablets) to validate features with real users.
- **AI/Chatbot Testing:** Validating prediction accuracy and response quality of the chatbot under diverse input conditions.

Automated test scripts and manual test cases are maintained to ensure consistent behavior after updates and new feature integrations.

2.2.4 Deployment Setup

The deployment of RadiantAura covers both mobile and web platforms:

- **Mobile App Deployment:** The Android version is prepared and released via the Google Play Store. The build process includes APK signing, versioning, and integration of Firebase for push notifications, authentication, and crash analytics.
- **Web Deployment:** The Laravel-based web platform is deployed to a production server, configured with HTTPS, user authentication layers, and database connectivity. Admin credentials and access roles are also secured.

- **Chatbot and Flask API Hosting:** The AI-powered chatbot and skin prediction model are hosted on a reliable server or cloud service (e.g., AWS, Heroku). The Flask API is configured with CORS settings to support both mobile and web frontend requests securely.
- **Deployment Documentation:** A comprehensive guide is included outlining steps to deploy new builds, update machine learning models, roll back changes, and monitor system health. It also includes a backup and recovery plan to prevent data loss.

2.3 Development Requirements

The development of a web application like RadiantAura requires a reliable and a well-equipped environment to support both frontend and backend development, database management, testing, and deployment. This section outlines the essential hardware and software resources necessary to ensure smooth development, efficient testing, and optimal performance across all components of the platform.

2.3.1 Hardware Requirements

Since RadiantAura is a cross-platform application system, end-users do not require any specialized hardware to access the platform, only a stable internet connection and a modern browser or an android phone. However, for the development team, adequate hardware is essential to ensure a seamless development experience, fast build times, and effective testing of real-time features such as messaging and face scanning. The following hardware specifications are recommended for development and testing:

2.3.1.1 Processor

Intel Core i5 or higher, Required for compiling Laravel and frontend assets efficiently.

2.3.1.2 RAM

Minimum 8 GB, Ensures smooth operation of IDEs, local servers, browsers, and testing tools simultaneously.

2.3.1.3 Storage

At least 7 GB of free disk space, needed to install development tools such as Laravel, Composer, and project files.

2.3.1.4 Display

Full HD display (1920x1080) or higher, for accurate UI/UX testing and layout verification across various screen sizes.

2.3.1.5 Internet Connection

Stable and high-speed internet, Essential for downloading dependencies, testing API calls, and ensuring real-time communication modules function correctly.

2.3.2 Software Requirements

To build, run, and maintain RadiantAura, a well-structured development stack is required. This includes a backend framework, database engine, frontend asset manager, and other essential tools to facilitate code writing, version control, and testing. Below is the complete list of software tools and technologies used in the project:

2.3.2.1 Laravel Framework

The core backend framework is used to build and structure the application. Laravel supports MVC architecture and provides built-in tools for routing, middleware, authentication, and database management.

2.3.2.2 PHP (v8.0 or higher)

Required to run Laravel efficiently and support its latest features and security updates.

2.3.2.3 MySQL / MariaDB

A relational database engine used to store all application data, including user profiles, job listings, application records, and admin logs.

2.3.2.4 VS Code

The preferred integrated development environment used for writing Laravel and frontend code, offering real-time debugging, extension support, and version control integration [4] [11].

2.3.2.5 Composer

A PHP dependency manager that allows for the installation and updating of Laravel packages and third-party libraries.

2.3.2.6 Android Studio

Android Studio with proper SDK configurations for the development of **RadiantAura** mobile app[12].

2.3.2.7 Operating System

Development can be performed on Windows 10/11, macOS, or any stable Linux distribution that supports the PHP and Laravel.

2.3.3 Server and Hosting Requirements

Most backend operations for RadiantAura are managed using a traditional web server setup with XAMPP and MySQL as the database management system. Server-side logic is handled using PHP and Laravel, ensuring efficient communication between the mobile app, chatbot, and web platform. APIs are developed in Laravel and integrated with both the Android app and the chatbot. For public access, the application will be published on the Google Play Store, making it available to a wide range of users. The web-based admin dashboard and API endpoints will be hosted on namecheap [13].

Chapter 3

3 Requirement Analysis

Requirement analysis for RadiantAura involves a thorough examination of user needs, technical feasibility, and market demands in the beauty and skincare industry. This process ensures the platform is designed to deliver personalized beauty recommendations, automated skin analysis, and real-time assistance through AI-driven solutions. By gathering insights through user surveys, expert interviews, and competitor analysis, the project identifies key features such as machine learning-based recommendations, image-based skin analysis, and a chatbot for real-time beauty advice. Additionally, the system prioritizes secure user authentication, data protection, and scalability to accommodate future growth. This analysis provides a clear understanding of the project's objectives, ensuring RadiantAura meets user expectations while delivering a reliable, user-friendly, and innovative beauty consultancy platform.

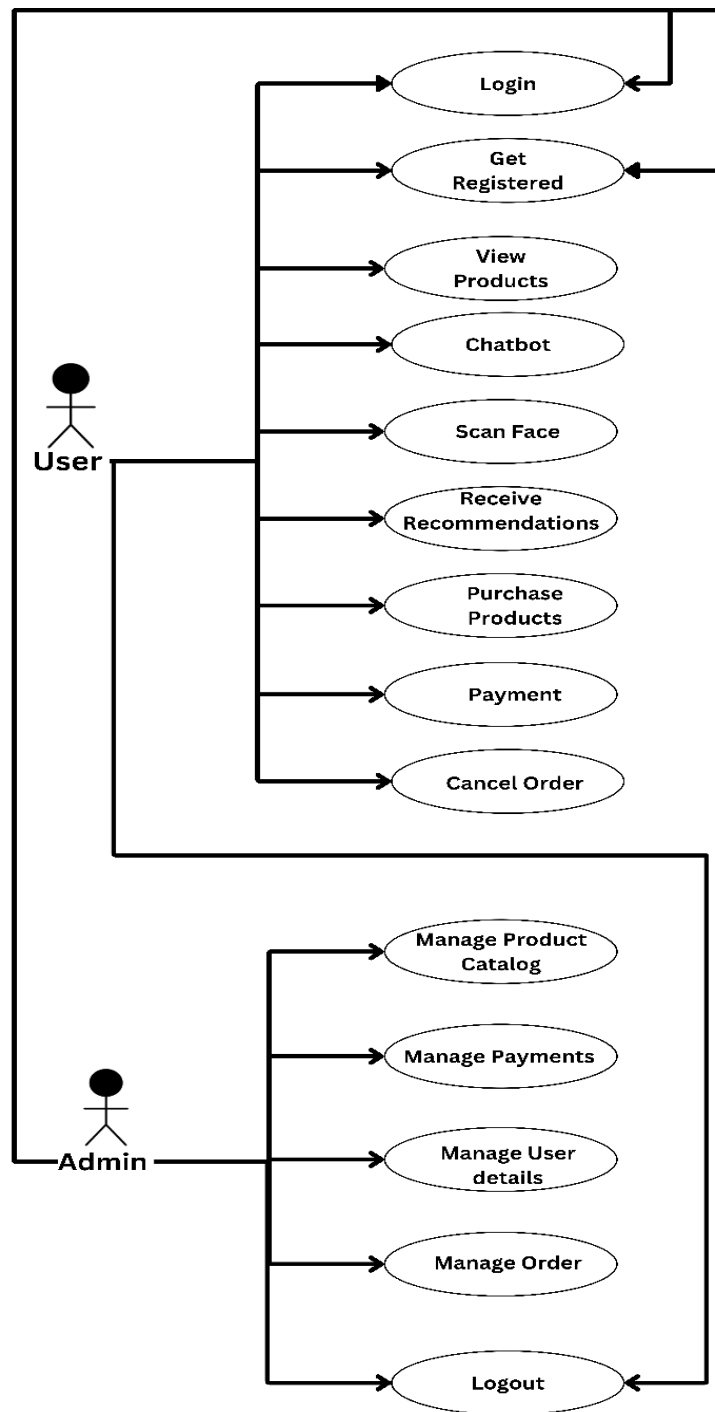


Figure 3.1: Use Case

3.1 Use Case Descriptions

The Use Case Descriptions provide detailed explanations of each action represented in the use case diagram. These descriptions help break down the system functionality into manageable tasks from the user's perspective.

3.1.1 UC-01: Authentication

This use case shows the authentication process for both Admin and User. **RadiantAura** provides secure login to access personalized features for personalized selections.

Table 3.1: Use Case Authentication

Use case id	01
Use case name	Authentication
Actors	• Admin • User
Description	A user of the System logs into the System.
Trigger	User requests to log in.
Pre-condition	The user has an account.
Post-condition	The actor is now logged into the system.
Normal flow	This use case starts when an actor wants to log into the system.
Alternative flow	If in the basic flow, the system cannot find the valid e-mail or the password is invalid, an error message is displayed.
Exceptions	Incorrect credentials. Request to re-enter the login details.
Business rules	The system must be connected to the network.

3.1.2 UC-02: Get Registration

This use case shows the Registration process for both Admin and User. **RadiantAura** provides secure signup to access personalized features for personalized beauty journey.

Table 3.2: Use case Get Registration

Use case id	02
Use case name	Get Registration
Actors	• User • Admin
Description	A user of the System creates an account.
Trigger	Select the register link
Pre-condition	When the user doesn't have an account to log in to the system.
Post-condition	The user's details are stored in the database.
Normal flow	• This use case starts when the user is not logged in to the system and goes to the login page. • The system prompts the user to send an e-mail and password or register a new account. • User selects the registration option. • The system displays the signup page that allows users to fill in required information • The user enters their information. • System verifies information and creates an account.
Alternative flow	Cancel Registration • The users select the cancel option. • The system returns the user to the home page without the user being logged in and any information entered has been erased. Invalid Information Entered • Users can enter information and submit it. • System displays a message to correct invalid information. • The user re-enters the information.
Exceptions	• If an existing user exists, then a "User already exists" message is displayed. • If information is missing "Insufficient Information" message is displayed.
Business rules	Active internet access is available.

3.1.3 UC-03: Choose options

This use case shows the product browsing process for User. **RadiantAura** provides interactive browsing process after secure login or signup.

Table 3.3: Use Case Choose options

Use case id	03
Use case name	View Products
Actors	User
Description	The user will view all the options and will choose.
Trigger	Users will choose any option.
Pre-condition	The actors log in to the system
Post-condition	The user opens an option
Normal flow	The users log in to the system. View the options
Alternative flow	The user did not register himself /herself and did not view the available options
Exceptions	The error occurs in the application. Server is down
Business rules	User registration is required.

3.1.4 UC-04: Chat

This use case shows the Chat process for Users to get concerned products and advices. **RadiantAura** provides personalized advices and recommendations based on users' skin type and their concerns providing them and easy, interactive, and enjoyable product selection process.

Table 3.4: Use case Chat

Use case id	04
Use case name	Chat
Actors	Customer
Description	The customer can go to chat box to describe his/her skin types and can get skin care advice.
Trigger	Customers want to get personalized recommendations.
Precondition	Customers must get registered on the app.
Post-condition	• Customer must know his/her skin type and skin problems. • Must Describes precisely.
Normal flow	• The customer successfully describes his/her skin type and skin problems. • Customers successfully get personalized recommendations.
Alternative flow	• Poor Skin Description. • Ambiguous recommendations.
Exceptions	• There is no internet. • An error occurs in the application. • Server down
Business rules	The request is rejected when the customer does not describe precisely.

3.1.5 UC-05: Scan Face

This use case shows the Face Scanning process for Users to get concerned products and advices. **RadiantAura** provides personalized advices and recommendations based on users' skin type and their concerns after scanning features.

Table 3.5: Use case Scan Face

Use case id	05
Use case name	Scan Face
Actors	User
Description	User scan his/her face to get automatic recommendations despite of describing it.
Trigger	Customers want to get personalized recommendations.
Precondition	The actors log in to the system.
Post-condition	The scan his/her face using device' camera.
Normal flow	System successfully scans face.
Alternative flow	The system does not scan face accurately.
Exceptions	• There is no internet. • An error occurs in the application. • Server down.
Business rules	User registration is required.

3.2 Functional Requirements

Functional requirements describe the expected behavior of the system, specifying what the system should do and how it should respond to user inputs. These include the core features of the system and its services, such as user registration, product browsing, cart management, order processing, and more.

3.2.1 User Registration and Authentication

The system must enable new users to register by providing their email, phone number, and a password. The registration process should ensure that the provided email or phone number is unique by verifying against existing records. Users will also be able to log into the system by using their email/phone number and password. Additionally, the system should provide an option for users to securely reset their passwords if forgotten, using a secure method such as an email or OTP verification. The login process should also allow users to securely log out of their session, ensuring that unauthorized access is prevented.

3.2.2 Face Analysis Module

A standout feature of RadiantAura is its intelligent Face Analysis Module, seamlessly integrated within the chatbot interface. This module enables users to perform real-time face scanning using their smartphone camera directly through the app. Once a user captures or uploads an image of their face, the system processes it using a custom-trained machine learning model hosted on the backend, which accurately detects the user's skin type such as oily, dry, combination, normal, or sensitive.

Based on the predicted skin type, the module dynamically generates personalized skincare advice, offering tips tailored to the user's skin condition. It also retrieves and displays a curated list of recommended products from the application's product database that are best suited to the identified skin type. This personalized feedback loop helps users make more informed decisions about their skincare routines, transforming the shopping experience into a guided, interactive, and results-driven journey.

By combining AI-powered skin analysis with real-time chatbot interaction, the Face Analysis Module bridges the gap between digital convenience and expert skincare consultation, delivering customized support that enhances user satisfaction and trust.

3.2.3 Personalized Recommendation Module

The Personalized Recommendation Module is a core feature of RadiantAura, designed to deliver AI-powered beauty suggestions tailored to each individual user. This module combines user preferences, such as product interests, skin concerns, and routine goals, with insights gained from the Face Analysis Module to generate highly relevant and targeted product recommendations.

Using intelligent algorithms and decision-making models implemented on the backend, the system analyzes both static user data (e.g., age, skin concerns, product usage history) and dynamic inputs (e.g., real-time skin type analysis). Based on this data, the module recommends skincare and cosmetic products that best suit the user's unique needs, ensuring more effective and satisfying results.

Additionally, the chatbot interface delivers these recommendations in a conversational and user-friendly manner, helping users understand why specific products were suggested and how they fit into their overall skincare routine. This module not only enhances the shopping

experience but also increases user engagement, trust, and retention by making every interaction feel curated and meaningful.

By combining personalization with intelligent product curation, RadiantAura transforms conventional browsing into an interactive, guided, and data-driven experience that aligns perfectly with modern user expectations in the beauty industry.

3.2.4 Cart Management

The product browsing and categorization feature of the RadiantAura is designed to offer users an intuitive and seamless shopping experience. Users will be able to explore a wide range of makeup products that are organized into clear categories, such as "Scrub", "Facewash", and "Foundation", making it easier for them to find products relevant to their specific needs. Within these categories, the system will enable users to filter products based on several criteria, including price, popularity, brand, ratings, and availability, allowing them to quickly narrow down their choices and find the best options within their preferred parameters. A search bar with autocomplete suggestions will further enhance the user experience by providing quick access to specific products as users type, ensuring that even when users are unsure of the exact product name, they can easily locate items that match their query. When a user selects a product, the app will display comprehensive details, including high-quality images, product pricing, an in-depth description, and usage tips, all of which are intended to assist the user in making informed decisions. These detailed product pages will provide all necessary information about the product's benefits, ingredients, and how to use it effectively, helping users feel confident in their purchase.

3.2.5 Chatbot

The system will include an AI-powered chatbot to provide real-time beauty advice and answer user queries by providing recommendations suitable for their skin types. It also detects skin types after analyzing skin concerns.

3.2.6 Admin Panel Management

The Admin Panel Management feature of the RadiantAura is designed to provide the system administrator with full control over the platform's operations, ensuring smooth management of products, orders, and user activities. The admin will log into the panel using secure admin credentials, which will be protected with advanced authentication mechanisms to ensure only

authorized personnel can access the system. Once logged in, admins will have a centralized dashboard where they can manage and oversee the core functions of the app. One of the key responsibilities of the admin is managing the product listings. The admin panel will allow admins to add, update, or delete products as needed, ensuring that the catalog remains up to date with new products, seasonal offerings, and any changes to existing items. This feature will provide a straightforward interface for updating product details such as names, descriptions, images, prices, and stock quantities. Admins will also be able to categorize products appropriately, ensuring that they appear in the correct sections for easy user navigation.

3.2.7 Product Browsing and Categorization

The product browsing and categorization feature of the RadiantAura is designed to offer users an intuitive and seamless shopping experience. Users will be able to explore a wide range of makeup products that are organized into clear categories, such as "Scrub", "Facewash", and "Foundation", making it easier for them to find products relevant to their specific needs. Within these categories, the system will enable users to filter products based on several criteria, including price, popularity, brand, ratings, and availability, allowing them to quickly narrow down their choices and find the best options within their preferred parameters. A search bar with autocomplete suggestions will further enhance the user experience by providing quick access to specific products as users type, ensuring that even when users are unsure of the exact product name, they can easily locate items that match their query. When a user selects a product, the app will display comprehensive details, including high-quality images, product pricing, an in-depth description, and usage tips, all of which are intended to assist the user in making informed decisions. These detailed product pages will provide all necessary information about the product's benefits, ingredients, and how to use it effectively, helping users feel confident in their purchase.

3.3 Nonfunctional Requirements

Nonfunctional requirements are those that establish parameters that may be used to evaluate the performance of a system and define criteria how well a system will work. Nonfunctional requirements are the limitations to which the system will work.

3.3.1 Performance

RadiantAura must offer a smooth, consistent, and responsive experience across all modules, whether it's browsing products, updating profiles, messaging, or managing admin tasks. The

platform should support real-time operations such as instant notifications, and live chat with minimal latency. Backend processes like API calls, server requests, and database queries must be optimized for speed, even as the user base and data volume grow. RadiantAura must maintain consistent performance standards during simultaneous multi-user activity, ensuring there is no perceptible lag or system degradation.

3.3.2 Security

Given that RadiantAura processes sensitive data, such as personal details, login credentials, and face scans security is a top priority. The platform must implement end-to-end encryption, HTTPS protocols, and secure password hashing (e.g., bcrypt). Laravel's built-in protections against CSRF, XSS, and SQL injection attacks should be utilized effectively. Role-based access control must be enforced to prevent unauthorized access, and additional layers like two-factor authentication may be implemented for enhanced security.

3.3.3 Scalability

As the platform gains traction, RadiantAura must be capable of scaling effectively to accommodate an expanding number of users, product listings, and system interactions. It should support both vertical scalability (enhancing system resources) and horizontal scalability (adding servers or microservices). Laravel's modular architecture and support for caching and queuing mechanisms make it a suitable choice for future upgrades. Scalability must apply not only to the user and admin dashboards but also to background services such as reporting, notifications, and anomaly detection, maintaining consistent performance under increased demand.

3.3.4 Accessibility

Ensure that the software is accessible to users, adhering to the latest web accessibility standards providing clear navigation.

3.3.5 Reliability

The system must be highly reliable in handling and processing all operations accurately. Whether it's submitting job applications, posting vacancies, or managing user data, the system must ensure data consistency and prevent corruption or loss. Features like automated testing, robust error handling, transaction rollback, and activity logging should be implemented to detect and recover from unexpected failures, ensuring system stability and operational trustworthiness.

3.3.6 Usability

The usability of RadiantAura is crucial to ensuring adoption and user satisfaction. The interface should be clean, intuitive, and accessible for users of all skill levels. Special attention must be given to UX design principles to reduce cognitive load and improve navigation. The system must be fully responsive, providing a seamless experience across desktops, tablets, and smartphones. Key features such as product browsing, chat, and face scanning should be easily accessible and logically placed within the user flow.

Chapter 4

4 Design and Architecture

This chapter explains how the different parts of the RadiantAura work together behind the scenes. The system has been carefully designed to be modular, scalable, and easy to maintain, so it can grow and improve over time without causing a problem.

It is built using a layered architecture and the Laravel MVC framework, the system separates concerns clearly, keeping the frontend (user interface) clean and responsive, while the backend (logic and data processing) handles everything from analyzing face to recommending the right products. Communication between the frontend and backend happens through a RESTful API, ensuring fast and reliable data exchange. The system creates a smooth and personalized experience for each user, whether they're scanning their face, chatting with the bot, or browsing skincare products.

4.1 System Architecture

The structure of the system explains the working of the system and describes the components of the project, their relationships, and how they interact with each other, with this architecture. This diagram shows the key components such as Database, Server, and a Client accessing them by an interface.

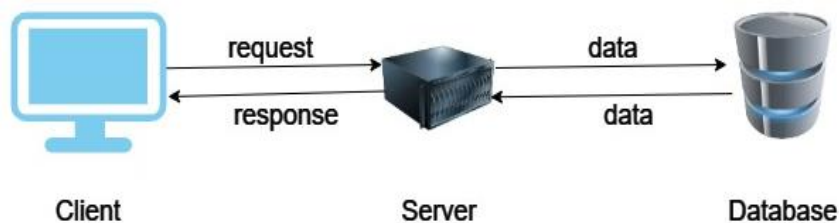


Figure 4.1: System Architecture

The RadiantAura platform is designed using a multi-layered architecture that ensures organized, secure, and efficient interaction among its two primary user roles: User, admin,. Each user type interacts with the system through a role-specific dashboard, built using modern frontend technologies such as HTML, CSS, JavaScript, and Bootstrap. These technologies offer a responsive and visually intuitive interface, allowing users to access relevant features based on their roles. Users search for products, scan face, and chat to get recommendations.

Admins interact through a dashboard that facilitates product listing, reviews incoming orders, and user profiles. The backend layer is powered by the Laravel PHP framework, which follows the MVC (Model View-Controller) architectural pattern. This layer handles all business logic and system operations through clearly defined controllers and middleware. Laravel's built-in middleware features enforce security by managing user authentication, session control, and protection against CSRF (Cross-Site Request Forgery) and XSS (Cross-Site Scripting) attacks. Key backend operations include processing search requests, scanning face, and chat with chatbot. At the base of the system lies the database layer, which is implemented using MySQL. This layer is responsible for structured data storage and efficient retrieval. It manages essential platform information such as user accounts (for both user and admin), product listings, user data, and orders. The clean separation between the layers allows for loose coupling and independent maintenance of each system component. This architectural design not only supports current functional requirements but also provides scalability and flexibility for future enhancements, including advanced analytics modules, recommendation engines, or third-party API integrations.

4.2 Data Representation

In the RadiantAura system, effective data representation is fundamental to delivering a seamless and intelligent skincare and beauty experience. The backend is powered by a relational database hosted locally on XAMPP, which ensures structured and consistent data storage. Core entities such as users, products, orders, chat messages, and skin analysis results are managed using well-defined MySQL database schemas, with normalized tables, foreign key relationships, and constraints that maintain data integrity and minimize redundancy.

The system's product database includes detailed fields such as product ID, name, category (e.g., skincare, makeup), skin type suitability, price, image path, and description. User tables store securely hashed login credentials, personal profiles, preferences, and purchase history. Orders are tracked through linked tables, capturing real-time transaction statuses like "pending," "shipped," or "delivered."

For the chatbot and face scanning module, the system also handles semi-structured data formats such as JSON. These are used for dynamic storage needs such as chatbot conversations, AI-

generated advice, face analysis results, and temporary logs, offering flexibility where rigid schemas are not required.

The hybrid data approach structured for transactional operations and semi-structured for flexible modules ensures the platform can scale efficiently while maintaining high performance. All sensitive data is stored with encryption protocols and follows data security best practices to protect user privacy and comply with safety standards.

Additionally, real-time interactions such as product recommendations and chatbot responses are optimized for fast querying and minimal latency. This balanced data architecture allows RadiantAura to support both its mobile app and web platform while enabling future expansion and integration with external APIs or third-party systems.

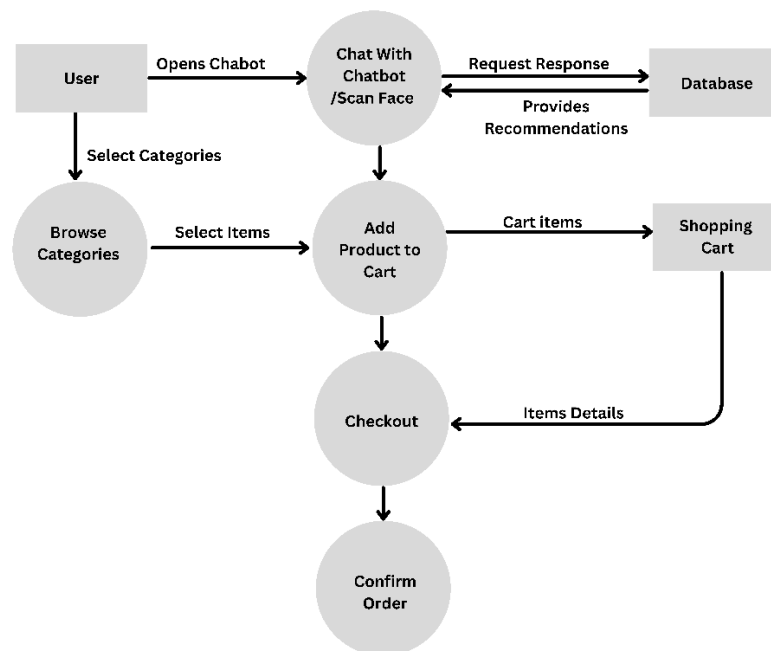


Figure 2.2: DFD for User

On the admin side, the data flow is structured to facilitate efficient management of RadiantAura's core operations. The admin panel interacts with the same centralized MySQL database, enabling administrators to perform CRUD (Create, Read, Update, Delete) operations on products, users, and orders through secured authentication mechanisms. Admins can view and manage product listings with attributes such as stock levels, categories, pricing, and promotional tags, which are dynamically fetched from and written to the product tables. User

management involves accessing encrypted credentials, tracking order histories, and handling customer inquiries submitted via chatbot logs. Real-time dashboards retrieve aggregated metrics like total sales, popular products, and user activity using optimized SQL queries and joins. The backend ensures that all admin interactions are logged for auditing and monitoring purposes, supporting accountability and data traceability. This streamlined admin data flow reinforces operational control, supports informed decision-making, and ensures the integrity and responsiveness of the overall RadiantAura platform.

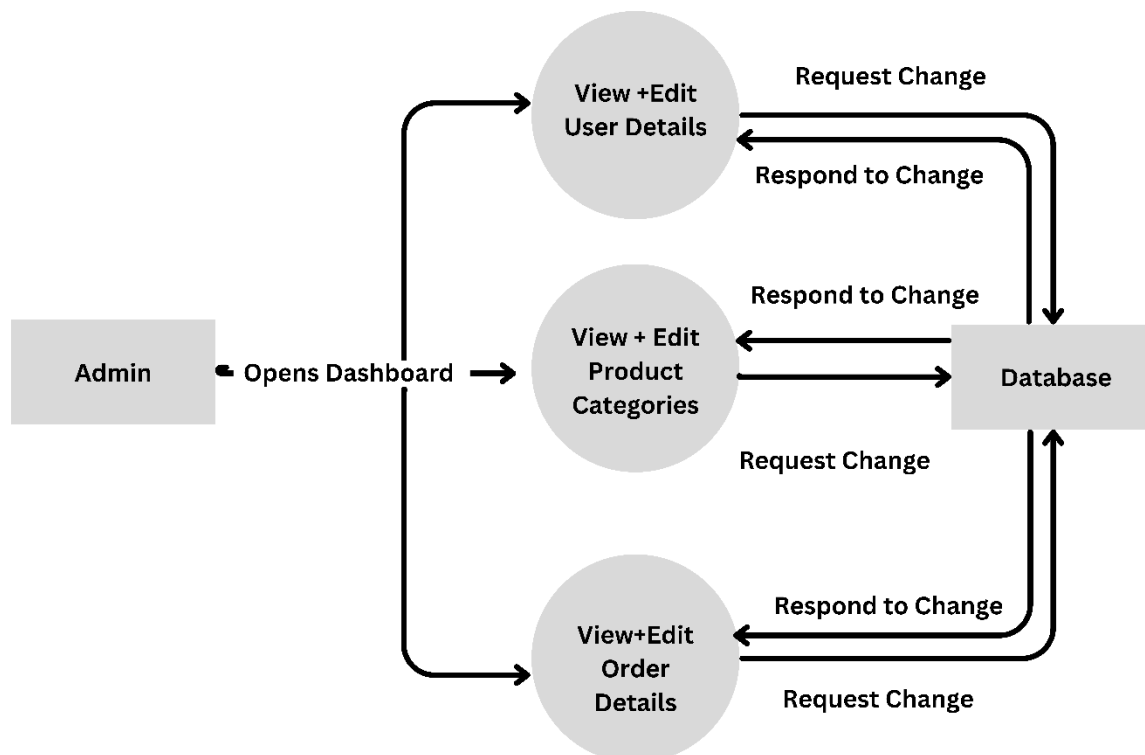


Figure 4.3: DFD for Admin

4.3 Process Flow/Representation

The RadiantAura platform adopts a structured, intelligent, and user-focused process flow that integrates real-time product discovery, personalized skincare advice, and secure shopping experiences across both mobile and web applications.

The process begins with user registration and login, where users authenticate via the mobile app or web platform. After successful authentication, users gain access to key features such as:

- **Browsing categorized beauty products** (e.g., skincare, makeup, face, eyes, lips)
- **Viewing detailed product descriptions** including skin compatibility, pricing, images, and recommendations

- **Adding products to a shopping cart**, modifying quantities, and proceeding to checkout
- **Receiving personalized skincare advice** from the AI-powered chatbot based on user preferences or face analysis

A unique feature of RadiantAura is the face analysis module, embedded within the chatbot interface. Users can capture a real-time image, and based on the analysis, the system provides insights such as skin type, skincare guidance, and recommended products from the database.

The backend, built using PHP and MySQL on XAMPP, manages the data securely through well-structured API endpoints. All user actions such as cart updates, order placements, and product searches interact with the backend through asynchronous AJAX or Retrofit-based API calls, ensuring real-time updates and seamless responsiveness.

For administrators, a separate web-based admin panel allows real-time product management, inventory control, and order tracking. Changes made by the admin, such as adding or editing products or updating order statuses, are instantly reflected across the system.

The chatbot system operates asynchronously, allowing users to receive dynamic, context-aware skincare advice without delay. Whether it's for product queries or skin consultation, the chatbot ensures ongoing user engagement and support.

This holistic and integrated process flow provides users with a smooth, personalized, and secure experience from facial scan and recommendation to product browsing and purchase.

4.3.1 Activity Diagram

An Activity Diagram is a visual representation in UML that illustrates the flow of operations or interactions within a system. It highlights the sequential logic of business processes or system functions using actions (activities), decision points, and transitions. The diagram typically begins with a start node and ends with a final node, effectively depicting the dynamic behavior of the system.

4.3.1.1 Activity Flow for User

The following activity diagram represents the primary user flow in the RadiantAura application, combining face scanning, AI-based chatbot interaction, product recommendations, cart management, and checkout processes.

Main Flow Includes

User Authentication

The process begins by checking whether the user has an existing account. If not, the user registers; otherwise, they proceed to login.

Home Interaction

Upon login, the user can navigate to different features, including Viewing and browsing categorized beauty products, Accessing the AI-powered chatbot for assistance or skincare advice, Using the face scanner to detect skin type and receive tailored recommendations.

Personalized Recommendations

After a successful face scan, the system suggests the user's skin type and provides product recommendations based on the results.

Shopping Flow

Users can view product details and add items to their shopping cart. Cart contents are displayed with options to change item quantities or remove items. When ready, users proceed to checkout.

Payment and Order Processing:

After confirming the cart, the user initiates payment. A validation check confirms whether the payment is successful. Upon approval, the system handles shipping and confirms the order.

This activity flow ensures a personalized, intuitive, and complete skincare shopping experience powered by real-time interaction, smart recommendations, and secure transactions.

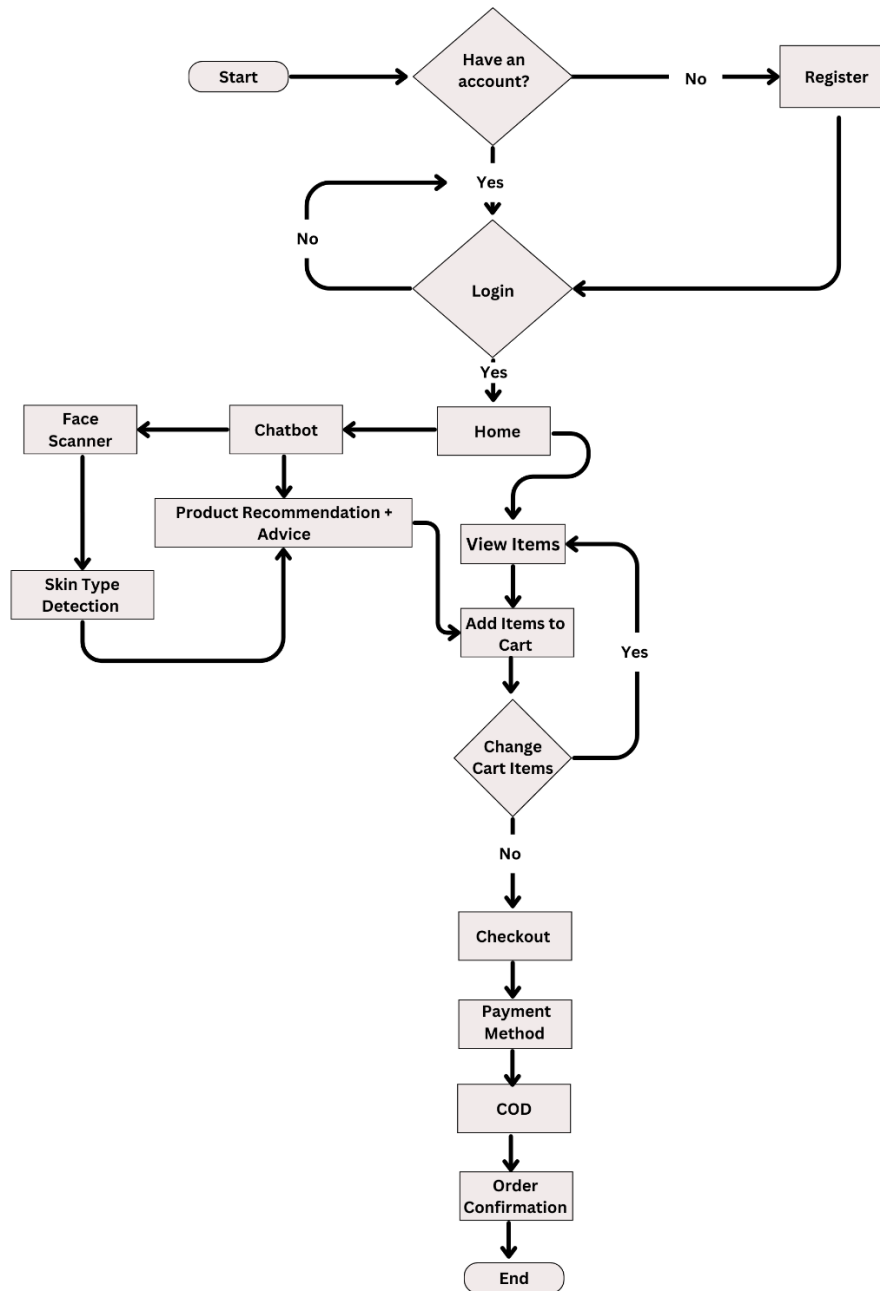


Figure 4.4: Process Flow Diagram

4.3.1.2 Activity Diagram for Admin

An activity diagram provides a visual representation of the flow of control or actions within a system. For the RadiantAura admin panel, the diagram illustrates how administrators interact with the system to manage essential operations like product listings, users, and order processing. It starts with user authentication and proceeds to administrative tasks, ensuring secure and structured management of the e-commerce platform.

- **Admin Authentication and Dashboard Access**

The process begins when an admin accesses the system. A decision node checks if the admin already has an account. If not, the system directs the user to the registration process. If an account exists, the admin proceeds to the login step. Upon successful authentication, the admin is directed to the dashboard (home).

- **Core Administrative Functions**

Once on the dashboard, the admin has access to three major functionalities: Admins can add new products, update existing product details (like name, category, price, image. Admins can view the list of registered users, update user roles (e.g., promote to moderator) or admin can remove suspicious users. Admins can monitor all orders, view payment and shipping details, update order status (pending, shipped, delivered), and handle customer service queries related to orders.

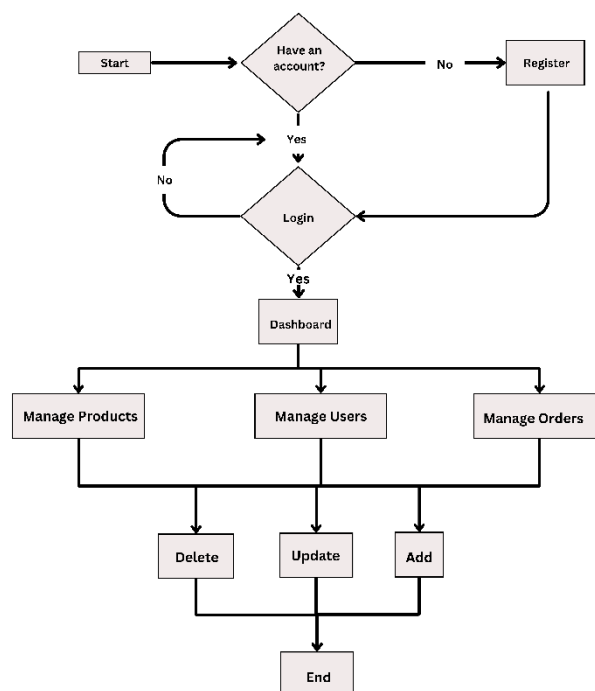


Figure 4.5: Activity Diagram for Admin

4.4 Sequence Diagram

A sequence diagram is a type of UML diagram that shows how objects interact in a particular system scenario. It represents the flow of messages and actions between different components

or users over time. In the RadiantAura project, sequence diagrams are used to visually describe the step-by-step interactions between the user and the system during key operations such as registration, login, browsing products, using Chatbot, Scanning face, adding products to cart, making payments, and logout. This diagram helps in understanding the flow of data and control, ensuring clarity in system design and smooth implementation of each feature. It gives a complete description of the working of RadiantAura.

4.4.1 Sequence Diagram (User Interaction)

The sequence diagram illustrates the flow of interactions between the user and the major components of the RadiantAura skincare system. It captures the dynamic behavior of the system in a time-sequenced manner, showcasing how a user navigates through the platform from registration to order confirmation.

4.4.1.1 Participants

- **User:** End-user interacting with the application.
- **Welcome Page:** Initial screen for login or registration.
- **Home Page:** Main product page post-login.
- **Chatbot:** AI-driven assistant to provide skincare advice.
- **Face Scanner:** Module that captures and analyzes facial data for recommendations.
- **Shopping Cart:** Section to hold selected products.
- **Checkout:** Interface to finalize the purchase.
- **Order Confirmation:** Displays order success and summary.

4.4.1.2 Flow Description

Authentication Process

User clicks “Continue to Register/Login.” Login or profile creation page is displayed.

User enters credentials. System authenticates and redirects to the Home Page.

Browsing Products

User can browse and explore product categories. Home page displays available skincare products.

Chatbot Interaction

User describes their skin problem. Chatbot analyzes input and shows product recommendations. Face scanner option is also provided.

Face Scan & Analysis

User scans their face via the face scanner module. Personalized product recommendations are displayed based on facial analysis.

Shopping Cart Actions

User adds a recommended product to the cart. System displays items currently in the cart.

Checkout Process

User clicks “Checkout.” A form is shown to input shipping/billing details. User enters the required information.

Order Confirmation

User reviews and confirms the order. System displays a confirmation message and summary of the purchase.

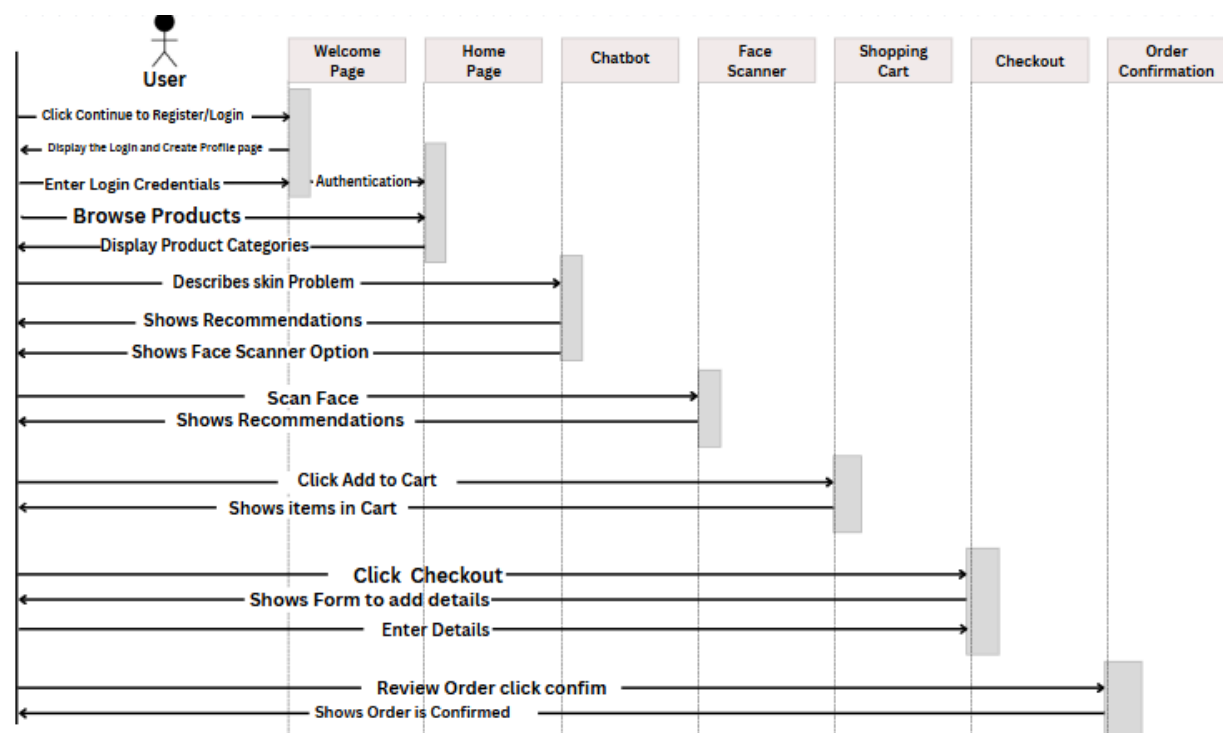


Figure 4.6: Sequence Diagram for User

4.4.2 Sequence Diagram (Admin Interaction)

The following sequence diagram outlines the step-by-step interactions between an Admin and the system components responsible for managing users, products, and orders. It reflects the administrative functionalities within the RadiantAura platform.

4.4.2.1 Participants

- **Admin:** The system administrator responsible for backend operations.

- **Login Page:** The entry point for admin authentication.
- **Admin Dashboard:** The central control panel post-login.
- **Product Management:** Module for adding, updating, or deleting products.
- **User Management:** Module to manage user accounts and roles.
- **Order Management:** Module to track and update order status.

4.4.2.2 Flow Description

Authentication

Admin accesses the login page. Enters login credentials. System verifies credentials and grants access to the Admin Dashboard.

Dashboard Access

Admin is presented with the dashboard. From the dashboard, the admin can choose to manage products, manage users, manage orders

Product Management

Admin navigates to the Product Management section. Actions include add new products (name, description, price, image), edit existing product details, delete discontinued or incorrect products. System confirms actions and updates the product database.

User Management

Admin accesses the User Management module. Views registered users. Has the ability to update user roles (e.g., make a user a moderator). Deactivate or remove users if necessary. System processes and stores these changes.

Order Management

Admin selects Order Management. Views recent orders and their statuses. Admin can approve or reject pending orders, update shipping status, view payment status. System reflects these updates in real time.

Logout

After completing tasks, Admin logs out from the system. Session ends and redirects to the login page.

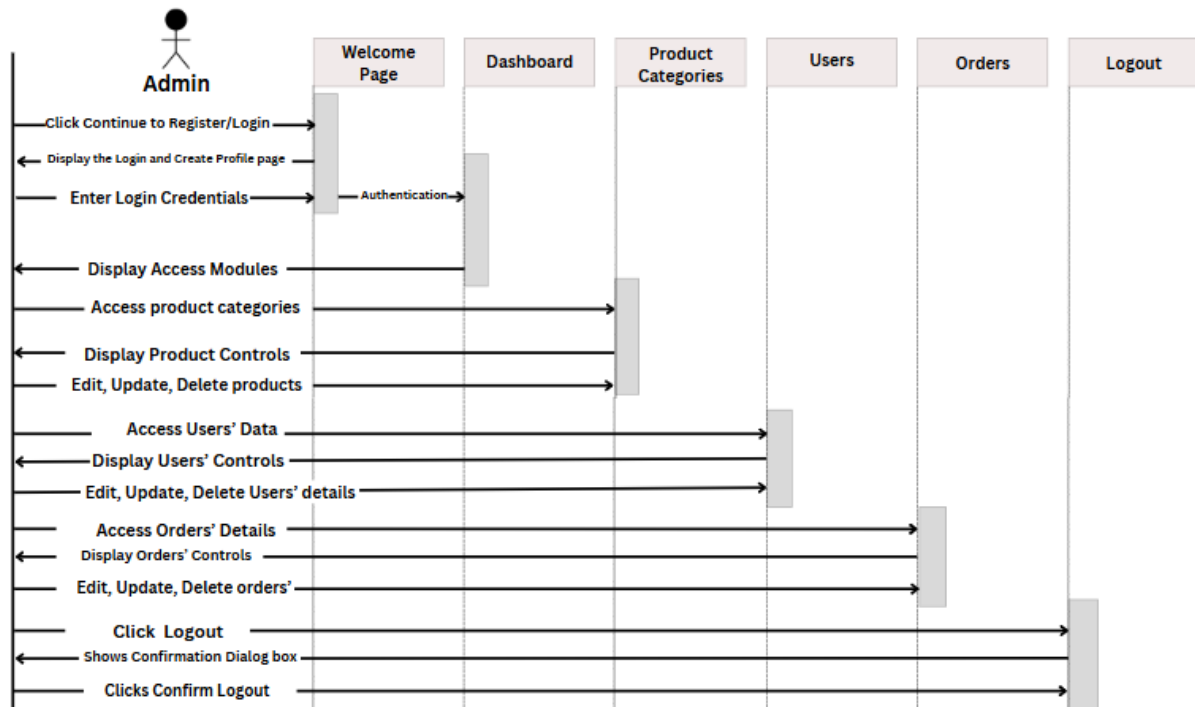


Figure 4.7: Sequence Diagram for Admin

4.5 Class Diagram

A class diagram is a visual representation used in software engineering to illustrate the structure of a system. It defines the classes (i.e., entities or objects), their attributes (data members), and the relationships between them. Class diagrams are a core part of the Unified Modeling Language and are essential for understanding how different components of an object-oriented system interact with each other.

The class diagram of **RadiantAura** represents the object-oriented structure of the system and defines the key classes, attributes, methods, and relationships among entities involved in the platform. The main module of the system is the Face Scanner, which scans the user's face to detect their skin type. Once the skin type is identified, it sends this information to both the Chatbot and the Server to recommend suitable products. The Chatbot is where users can type questions or ask for suggestions. It takes the input, processes it, and gives a helpful response. It works closely with the Face Scanner and the Server to make accurate suggestions.

The Server connects everything. It pulls product data and sends recommendations to both the chatbot and the face scanner. Products are stored in the Product class. Each product has details like name, price, and description. Products are connected to the chatbot, face scanner, and the shopping cart, because they need to be shown to users in different scenarios. When a customer

decides to buy something, their chosen products go into the Shopping Cart. The cart stores all the product details and passes them on to create an Order. Every Order is placed by a customer, who has their own profile with contact information. After the order is placed, it also has a Payment class where the total cost, payment status, and other details are handled.

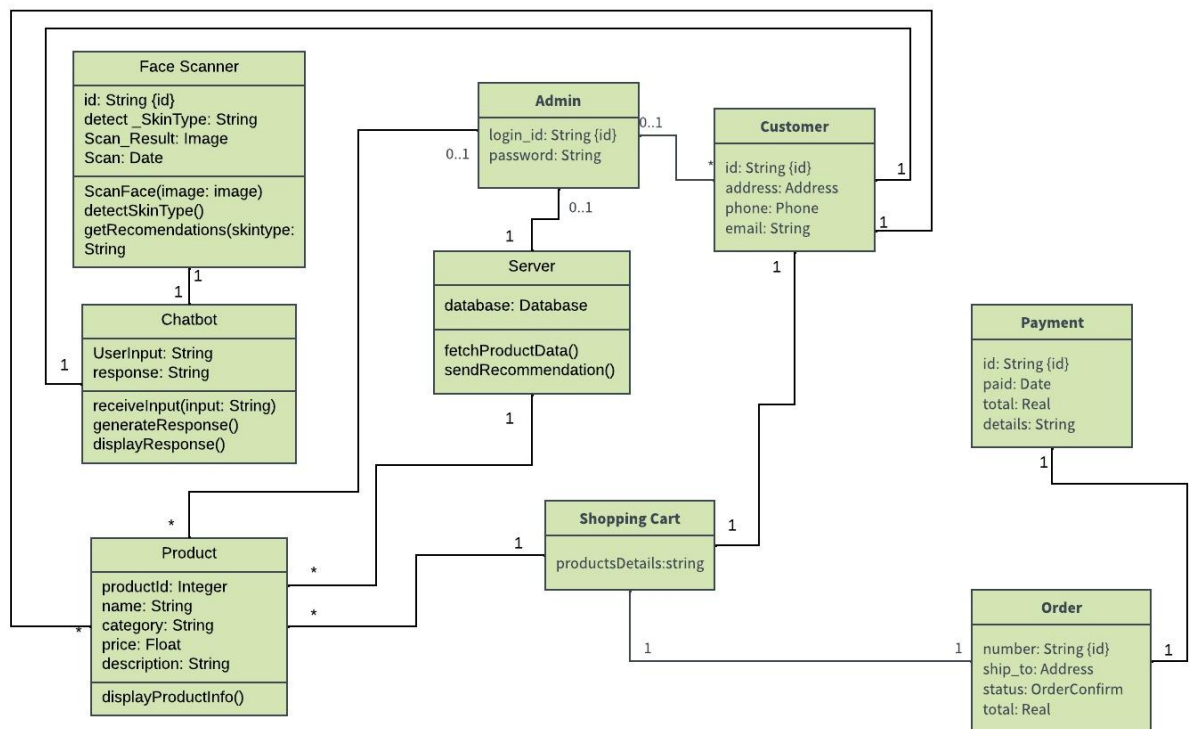


Figure 4.8: Class Diagram

Chapter 5

5 Implementation

In the implementation phase of RadiantAura, the system is developed using Kotlin for the Android application and PHP, HTML, CSS, and JavaScript for the web-based admin panel. The platform is designed to function seamlessly across Android devices and web browsers, ensuring a responsive and user-friendly experience. The goal is to provide users with personalized beauty advices, including face analysis, AI-driven recommendations, and secure user interactions.

5.1 Laravel Framework and the MVC Architecture

The RadiantAura is developed using the Laravel framework, following the Model-View Controller (MVC) design pattern. This approach ensures a clean separation between business logic, user interface, and data management.

Implementation Flow:

- **Routing & Controllers**

Laravel routes incoming HTTP requests to specific controller functions based on the user's role and action (e.g., applying for a job, updating a profile, shortlisting a candidate). RESTful route structures were maintained for consistency and API readiness.

- **Models & Eloquent ORM**

Data entities such as users, admins, responses, messages, and profiles are represented as models. Eloquent ORM is used to manage relationships and query the database efficiently.

- **Blade Templating Engine (Views)**

The presentation layer was implemented using Laravel Blade templates. This allowed the development team to create reusable UI components for dashboards, forms, listings, and cards, reducing frontend redundancy.

- **Middleware**

Role-based middleware was implemented to control access at route level. For example, only admins can manage user accounts and products. This architectural model promotes scalability, maintainability, and logical code separation throughout the development lifecycle.

5.2 Development Tools and Technologies

5.2.1 Frontend

5.2.1.1 HTML5, CSS3, JavaScript

These core technologies provided the foundation for building responsive, accessible, and interactive web interfaces[1].

5.2.1.2 Bootstrap 5

Bootstrap was used for creating mobile-first, responsive layouts, reducing development time through ready-to-use UI components[14].

5.2.1.3 Blade Templating Engine

Enabled dynamic rendering of HTML templates with Laravel's backend logic.

5.2.1.4 XML

Used to design layouts for the android version of RadiantAura using Android Studio.

5.2.1.5 Figma/Canva

Used for designing UI/UX wireframes and graphics. Figma offered real time collaborative design, while Canva was used for aesthetic illustrations and branding assets[15] [16].

5.2.2 Backend

5.2.2.1 Laravel (PHP Framework)

Laravel was selected due to its extensive documentation, security features (e.g., CSRF protection, hashing), RESTful API support, and built-in functionalities like email notifications, file storage, and middleware[4] [17].

5.2.2.2 Composer

Used for managing Laravel dependencies and PHP packages efficiently[18].

5.2.3 Database

5.2.3.1 MySQL

A reliable, open-source RDBMS chosen for its compatibility with Laravel's Eloquent ORM and ability to handle structured data efficiently[3].

5.2.3.2 phpMyAdmin

A graphical interface used to interact with the MySQL database during development, helpful for viewing tables, executing queries, and backing up data[19].

5.2.4 Other Tools

5.2.4.1 XAMPP / Laravel Valet

Used for local development and testing before deployment[19].

5.2.4.2 Visual Studio Code (VS Code)

A modern and lightweight code editor with extensive plugin support for PHP, Laravel, Blade, and Git[11].

5.3 User Registration Algorithm

The user registration algorithm handles the account creation process. It allows new users to register easily and securely using their email and password. These credentials are stored securely by applying hashing over the password to protect data from breaches. This algorithm verifies user credentials, manages session handling, and stores user information securely in the MySQL (for the admin panel).

5.4 Face Analysis and Recommendation Algorithm

Face Analysis Module is the core of this project RadiantAura. User interacts with it through chatbot. When user activates it, it scans the user's face, detect skin type, and then request for specific recommendations from backend. It securely performs scanning, skin type prediction, and shows recommendations making the beauty journey more enjoyable and efficient.

5.5 Security and Data Protection

Security is a priority throughout the implementation phase. The system employs encryption techniques to protect sensitive data, including user profiles. It provides end to end privacy by applying security checks e.g. password hashing, and sessions. Secure user authentication methods ensure data privacy, and session management protects user information from unauthorized access.

5.6 Admin Panel

The administrator panel acts as the control center of the RadiantAura system, responsible for enforcing rules, moderating content, and maintaining platform integrity.

5.6.1 Admin Capabilities

This portion tell what admin can do basically the authorities of admin.

5.6.1.1 User Management

Admins can view, edit, block, or delete user accounts. Users are categorized by role, and permissions are enforced accordingly.

5.6.1.2 Catalog Management

Job posts are reviewed for quality and authenticity before being published. Admins have the authority to reject inappropriate or suspicious listings.

5.6.1.3 Dashboard Analytics

Admins can access platform statistics such as a number of active users, orders placed, catalog trends and application trends, allowing for informed decisions and future planning.

This centralized panel ensures the smooth operation of the platform and supports the admin in maintaining a safe and professional environment.

5.7 Application Integration and Optimization

To ensure long-term performance and discoverability, several backend integrations and enhancements were implemented:

5.7.1 SEO

Meta tags, clean URLs, and structured content improve website's visibility on platforms like Google.

5.7.2 Database Optimization

Laravel Eloquent queries are optimized using eager loading and indexing to minimize load times and improve scalability.

5.7.3 Security Measures

Passwords are hashed, CSRF tokens are enabled, and all inputs are validated to prevent XSS and SQL injection attacks.

5.7.4 Responsive Design

All dashboards and forms are responsive, ensuring accessibility across desktops, tablets, and smartphones. These enhancements ensure a reliable and accessible experience for all user types.

5.8 Deployment Details

5.8.1 Hosting Environment

The Radiant Aura project (Web and Android app) was deployed using a shared hosting server provided by Hostinger, which supports PHP 8.x and MySQL. A custom domain was configured with SSL (Let's Encrypt) to ensure secure access via HTTPS for the web application. The Android app communicates securely with the web server API.

5.8.2 Backend Deployment

The Laravel web application was deployed using FileZilla (FTP). The .env file was updated with live server credentials[13]. Essential Artisan commands were executed to ensure proper application configuration and migration:

```
bash
```

```
CopyEdit
```

```
php artisan migrate --force
```

```
php artisan config:cache
```

```
php artisan route:cache
```

5.8.3 Database Configuration

The live MySQL database was configured through phpMyAdmin. Laravel migrations were run to apply the necessary schema, and the database access credentials were set in the .env file. This database is shared between the web and mobile applications.

5.8.4 Mail Setup

SMTP mail settings (using Gmail SMTP) were added to the .env file to enable email notifications and form submission handling in the web interface.

5.8.5 Android App Deployment

The Radiant Aura Android application was built in Kotlin and communicates with the Laravel backend using Retrofit APIs. The app consumes data such as skin type predictions, skincare advice, and product recommendations from a Flask-powered AI microservice and the Laravel backend. The app was signed and built as an APK/AAB for distribution[20].

5.8.6 Version Control

All source code (Laravel web app, Android app, and Flask API) was managed using Git & GitHub for version control, collaboration, and future maintenance.

5.9 User interface

The RadiantAura user interface is designed for ease of use and intuitive navigation. The Android application allows users to access the face analysis, product recommendations, and chatbot features seamlessly. The admin panel provides administrators with tools to manage user accounts, and product catalogs, ensuring smooth platform operation.

5.9.1 Welcome Screen

The welcome screen utilizes a warm greeting to new users, highlighting the software's purpose of providing both ease of use and a deeper understanding of its functionalities. It presents clear calls to action, inviting users to begin their journey within the application.

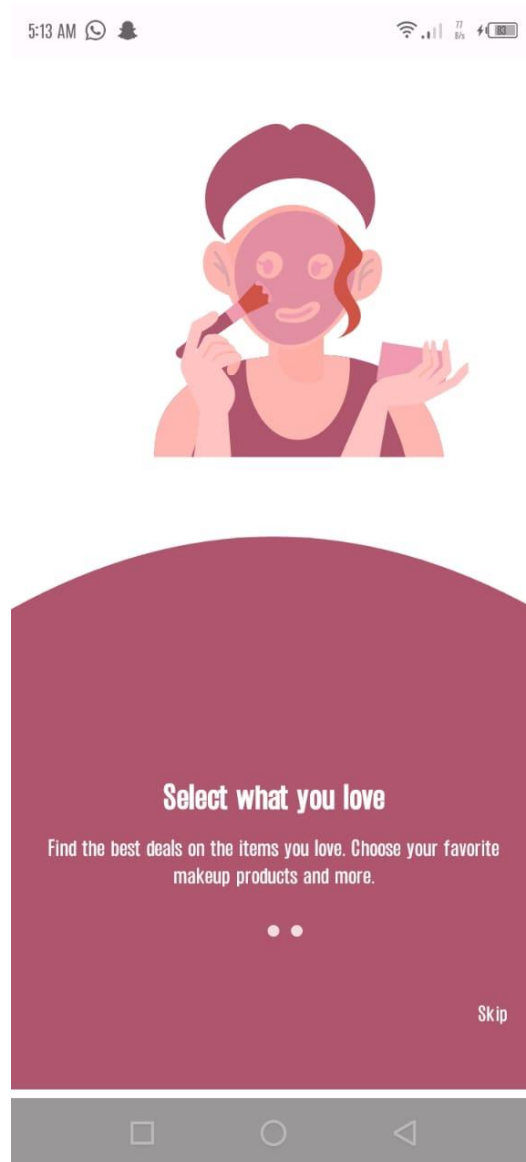


Figure 5.1: App Welcome Screen

5.9.2 RadiantAura Features

The image depicts the home screen of a RadiantAura mobile. The navigation bar clearly communicates the app's features: Chat icon for personalized recommendations through chat and inside it there is another module face scanner is integrated, cart, wishlist, and settings to manage profile. At the top left corner. There is a menu button which encloses the available products' categories.

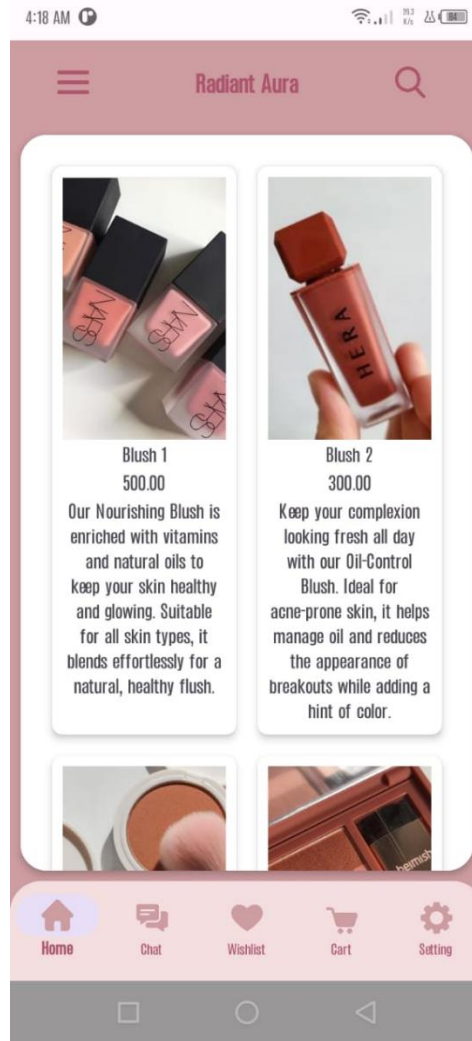


Figure 5.2: App HomePage

This screen showcases a user-friendly interface that allows users to chat with RadiantAura. The send button shows its functionality to send user query to the backend, while the camera icon opens a gateway to scan face for both skin type detection and products' recommendations. Ask questions, seek information, and explore your concerned products effortlessly" expands on the app's capabilities and potential benefits.

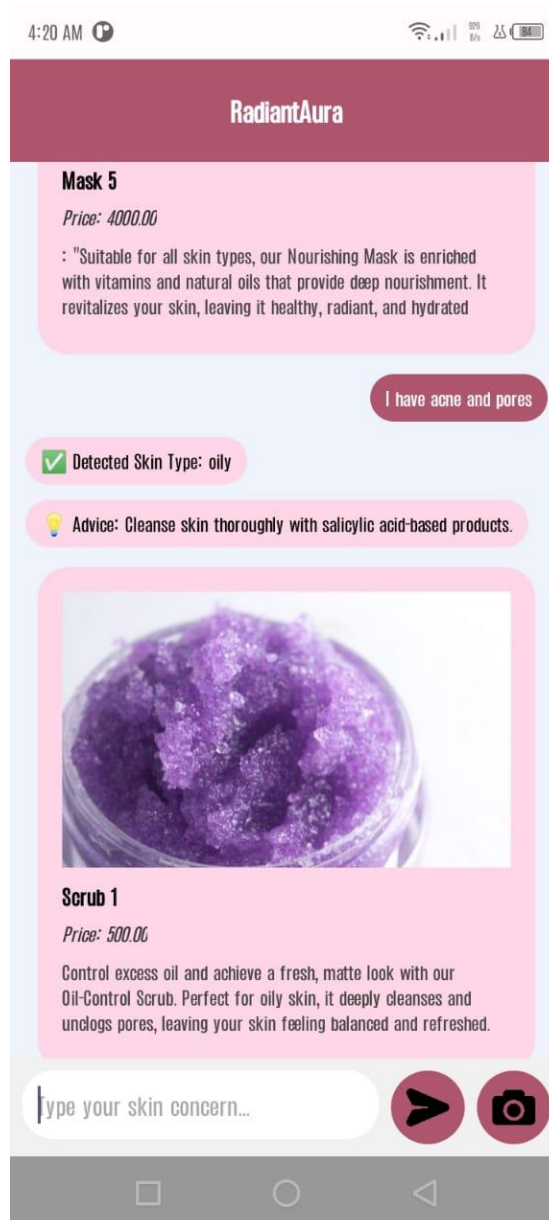


Figure 5.3: App Chatbot

This screen depicts a user interface for a face scanner integrated in **RadiantAura** Website and mobile app. Its icon clearly shows its functionality, inviting users to tap it for their skin care journey through skin type detection along with suggestions. When user clicks the “take picture” button, scanner gets closed and start scanning. After successful scan it shows skin type, products, and advice in chatbot.

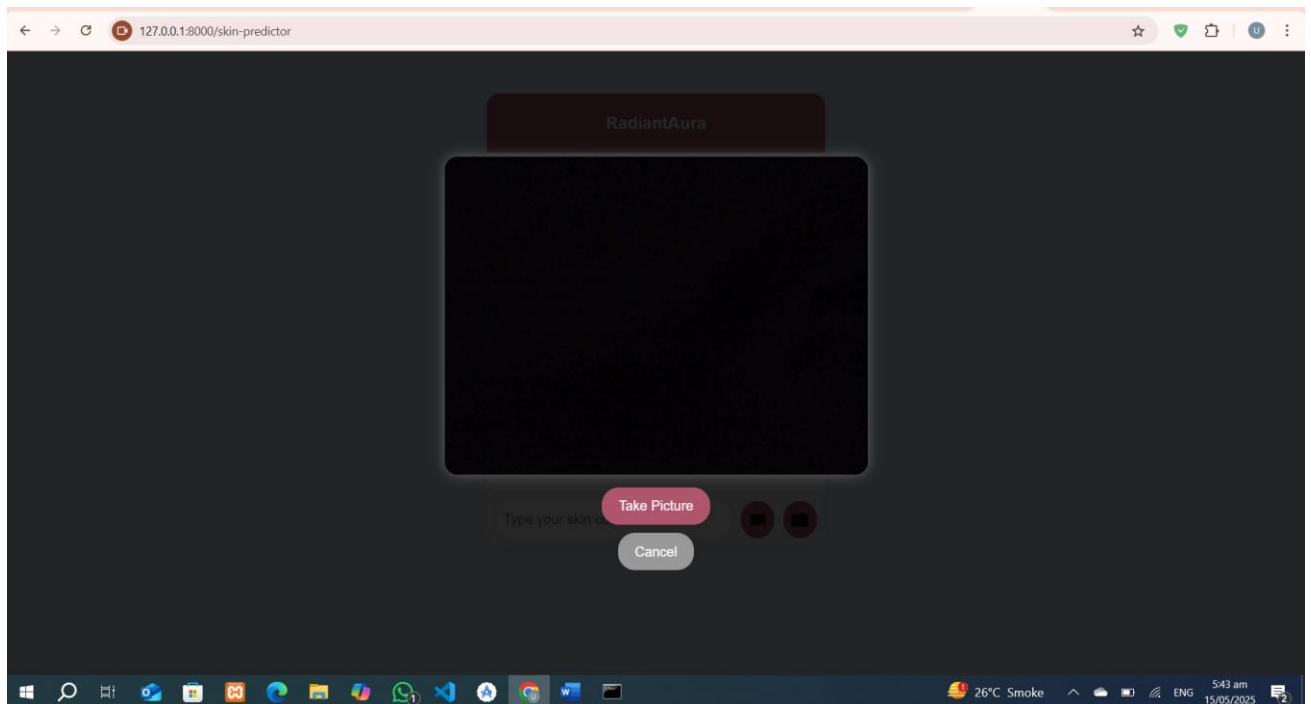


Figure 3: Web Face Scanner

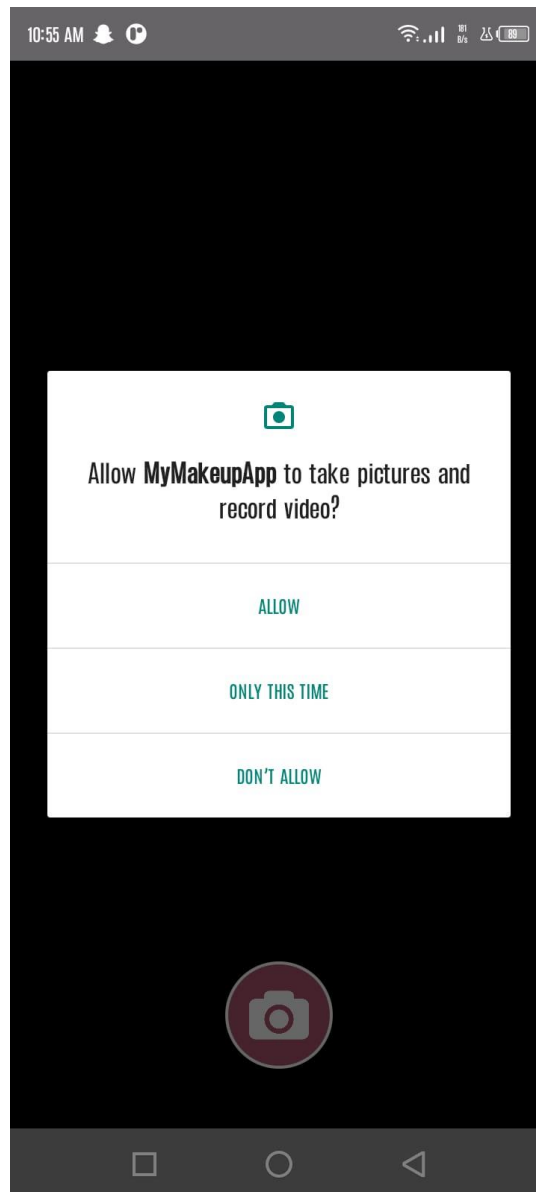


Figure 5.5: Face Scanner in App

5.9.3 Sign Up Page

The image depicts a sign-up screen for a new user. The bolded heading “Create Account” guides users through the account creation process.

The screen prompts users to enter their name, username/email address and password, following standard secure sign-up practices. Below those sections, there’s a link labeled “Already have an account? Login,” which provides a clear path for existing users to access their accounts. Finally, a prominent “Sign Up” button allows users to submit their information and create a new account.

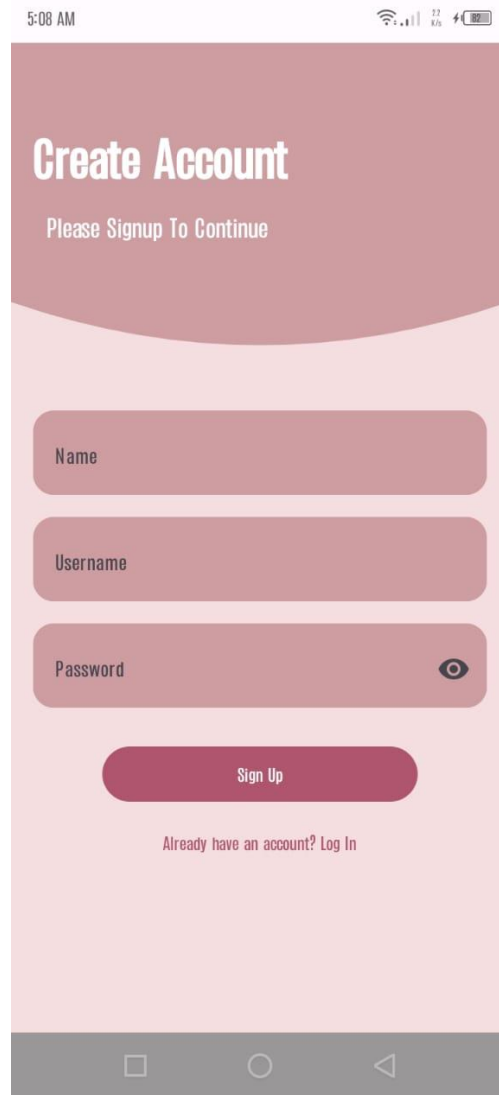


Figure 4 SignUp Page

5.9.4 Login Page

This screen depicts a login screen for a secure application. This indicates a well-established application. Below that are clearly labeled fields for "Username" and "Password", allowing users to easily enter their credentials. Additionally, a "Forgot Password" option provides a safety net for users who may have forgotten their login information. Finally, prominent "Login" and "Sign Up" buttons offer clear calls to action, inviting users to either access their accounts or create new ones.

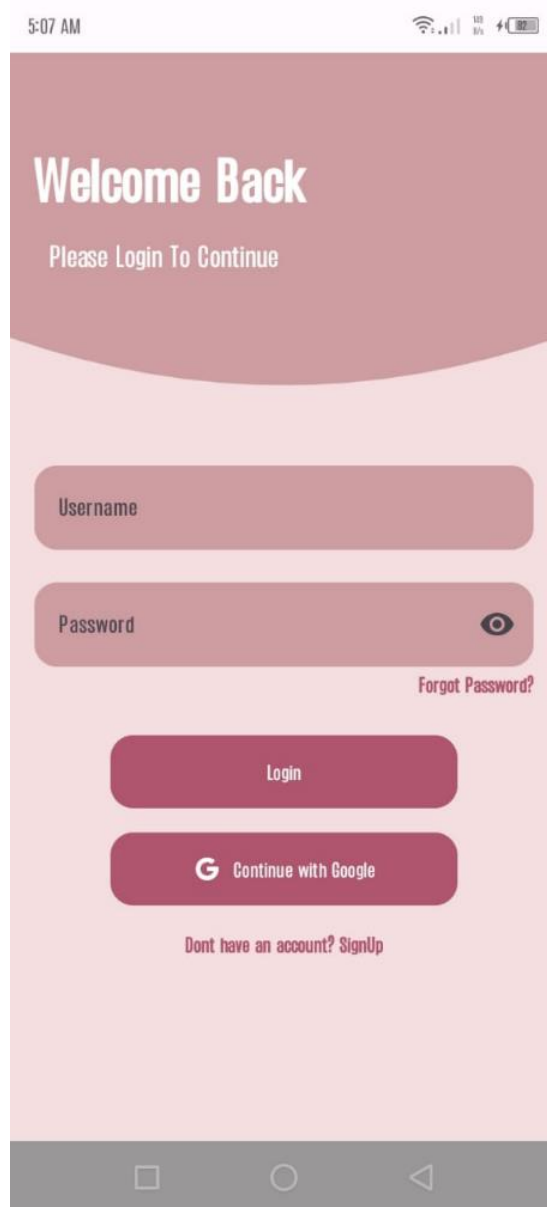


Figure 5: Login Page

5.9.5 Home Screen

The image depicts a Home screen for **RadiantAura** website. At the top in header section there are login, signup, and cart options to register, login back into accounts, and to add items to cart. Below are the categories of available products' recommendations. There is a chat icon in the bottom right of the page, this suggests the website allows users to communicate and get personalized recommendations about their skin care and products' selection.

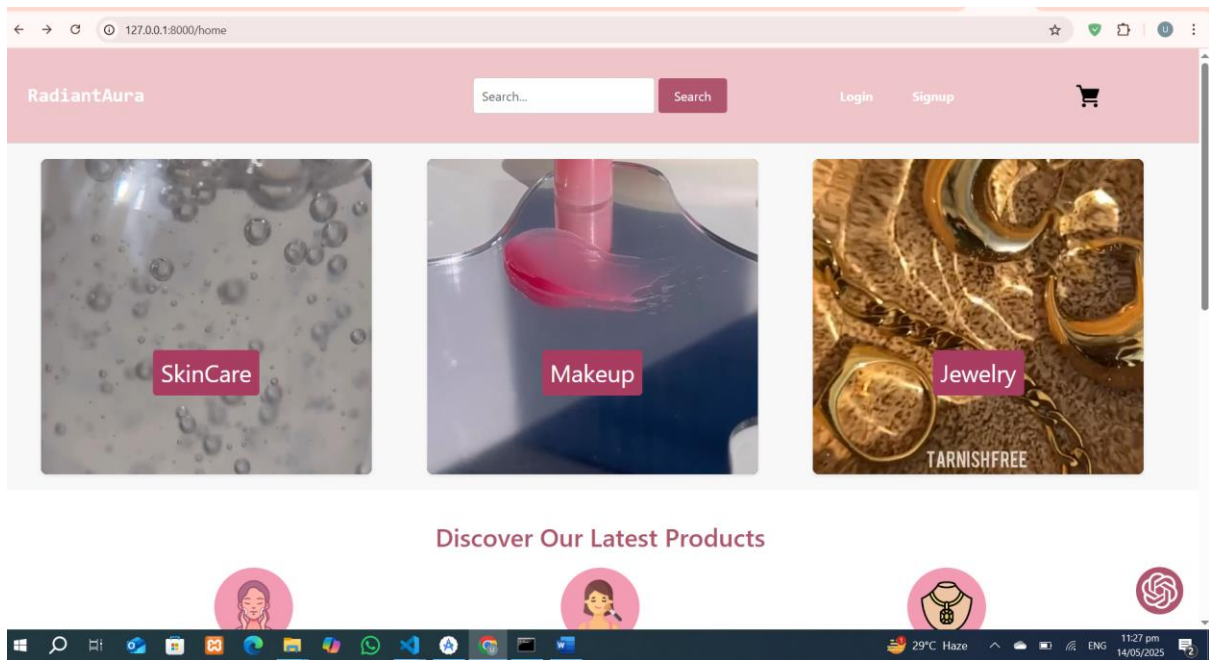


Figure 5.8: RadiantAura Website Homepage

5.9.6 Product Categories

The Radiant Aura platform features a well-organized Skincare Categories section to help users quickly explore products and advice tailored to their specific skin needs.

In the Android app, categories are displayed in a vertical sidebar for easy navigation. This layout allows users to browse through various skincare types without leaving the current screen, providing a smooth and intuitive mobile experience.

In the web application, skincare categories are presented in a horizontally scrollable bar at the top or middle of the page. This design enhances visibility and encourages exploration by allowing users to scroll through categories effortlessly.

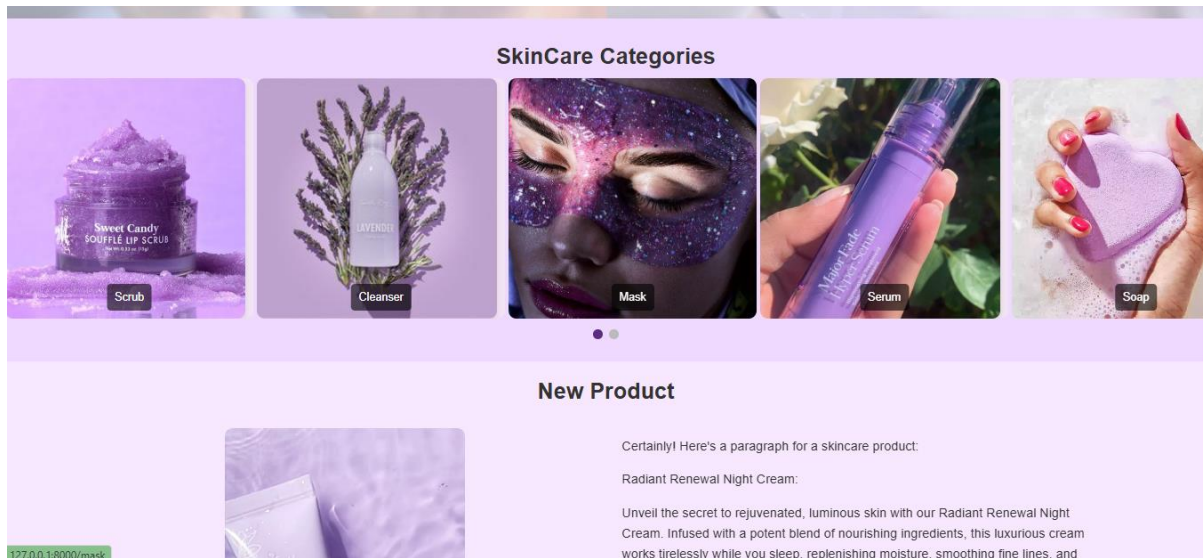


Figure 6: Product Categories

5.9.7 Admin Panel

Admin Panel is core of any website. RadiantAura has a web-based admin panel which is connected to both website users' side and mobile application. It manages products categories and their sub-categories. Admin panel keeps track of all registered users and has ability to remove any user. It also manages order details.

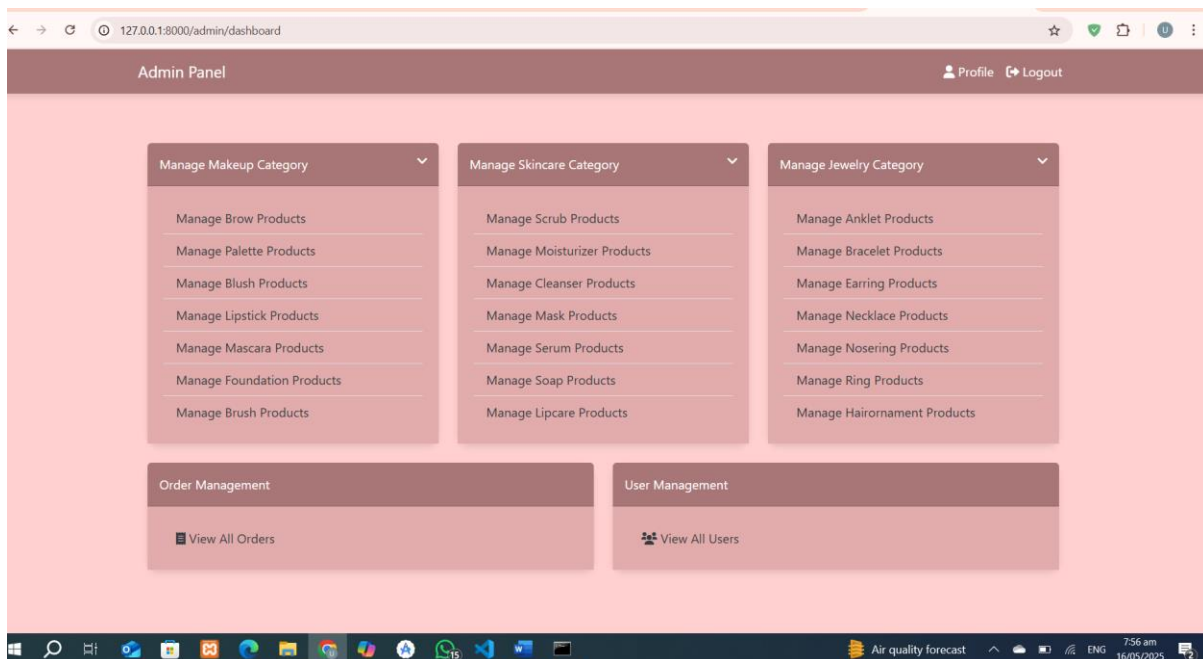


Figure 7: Admin Panel

At RadiantAura, we offer a complete range of skincare products designed to nourish and enhance your natural beauty. Start your routine with our gentle cleansers that remove dirt and

impurities while keeping your skin balanced. Follow with our exfoliating scrubs, perfect for smoothing away dead skin cells and revealing a fresh glow. Treat your skin to a boost with our deeply hydrating masks, formulated to rejuvenate and refresh. Target specific skin concerns with our powerful serums, packed with active ingredients to brighten, firm, and even out your skin tone. Complete your care with our soft, nourishing soaps, ideal for daily cleansing and leaving your skin feeling clean, soft, and delicately scented. Each product is crafted with care to give your skin the love it deserves.

Chapter 6

6 Testing

Testing is a critical phase in the software development lifecycle that ensures the application behaves as expected, is user-friendly, and meets its intended requirements. It helps identify bugs, verify functionalities, and validate that the user experience is smooth and reliable.

For this project **RadiantAura** testing focused on verifying the accuracy of real-time skin analysis using a face scanner, the responsiveness of the chatbot, and the relevance of product suggestions.

6.1 Unit Testing

Unit testing is a foundational aspect of the RadiantAura development process, focusing on the validation of individual components in isolation to ensure their correctness and reliability. This testing was carried out using Laravel's PHPUnit framework, which provided a robust and automated way to verify the behavior of backend logic, controller methods, and data-handling functions.

The purpose of unit testing was to catch potential issues early in the development cycle by testing the smallest units of functionality such as user authentication, product CRUD operations, chatbot response handling, and order processing routines in a controlled environment. Each function and method were tested with both valid and invalid input to check for expected outputs, error handling, and data integrity. For instance, authentication tests ensured that only valid credentials could log a user in, and that incorrect login attempts were properly rejected with informative feedback.

Tests were also created to simulate database interactions using Laravel's in-memory SQLite or testing database, verifying that data insertion, updates, deletions, and queries behaved correctly and did not produce unintended side effects. Controller methods were tested to ensure they returned accurate HTTP responses, handled exceptions gracefully, and correctly interfaced with models and services.

By conducting comprehensive unit testing, the development team was able to significantly reduce bugs, improve code quality, and enhance maintainability. It also provided a safety net

during later development stages, allowing for confident refactoring and integration of new features without breaking existing functionality. Overall, unit testing played a key role in ensuring that the core components of RadiantAura performed accurately, securely, and consistently.

Table 6.1: Unit Testing

Test Case ID	Test Case Name	Test Description	Expected Result	Actual Result
UT-001	Live Face Capture	Capture a live image using webcam/camera.	Camera activates and captures clear facial data.	Live image captured successfully.
UT-002	Skin Type Detection	Analyze captured image in real-time.	Skin type is correctly detected from live image.	Skin analysis accurate and consistent.
UT-003	Chatbot Response Logic	Enter a sample question in chatbot window.	Chatbot returns a relevant, immediate reply.	Chatbot responded correctly.
UT-004	Product Recommendation	Trigger product suggestions from current skin analysis.	Suitable products recommended on-the-fly.	Real-time suggestions shown.

6.2 Manual Testing

Manual testing was conducted thoroughly throughout the development cycle of RadiantAura to ensure that all features functioned correctly and delivered a seamless user experience. This type of testing involved human testers interacting with the application directly via the user interface, simulating real-world scenarios and use cases that end-users would encounter.

Testers manually navigated through various sections of the application including login and registration, product browsing, chatbot interaction, face scanning, shopping cart functionality, and the checkout process to verify that each component responded as expected. Special attention was paid to UI responsiveness, button functionality, form validations, error messages, and navigation flows to detect any anomalies or broken links.

Unlike automated testing, manual testing allowed testers to assess aesthetic elements and usability aspects, such as layout alignment, content clarity, image rendering, and overall visual

consistency across devices and browsers. This helped uncover issues that automated scripts might overlook, such as text truncation, layout breaks, or inconsistent element spacing.

Test cases were executed systematically, with detailed documentation of inputs, expected outcomes, actual results, and any discrepancies found. When bugs or inconsistencies were identified, they were reported immediately to the development team for resolution. Manual regression testing was also performed after code updates to ensure that new changes did not adversely affect existing functionality.

Overall, manual testing played a critical role in quality assurance by complementing automated testing methods, validating user-facing features, and ensuring RadiantAura delivered a polished, functional, and user-friendly interface.

Table 6.2: Manual Testing

Test Case ID	Feature	Test Scenario	Expected Outcome	Actual Outcome
MT01	Camera Activation	Start scanner and allow camera access.	Camera activates instantly.	Camera worked as expected.
MT02	Real-Time Skin Analysis	Detect skin type immediately after live scan.	Skin profile shown without delay.	Skin type displayed accurately.
MT03	Chatbot Interaction	Ask various skin-related questions.	Bot replies with relevant answers.	Chatbot gave accurate answers.
MT04	Recommendation Display	View product recommendations based on detected skin.	Products shown match current skin type.	Recommendations were appropriate.

6.3 Functional Testing

Functional testing for RadiantAura was carried out to ensure that all features and modules operated in strict accordance with the defined functional requirements and business rules. The

primary objective of this phase was to validate that the software system delivered the expected output when provided with specific inputs, thereby confirming its readiness for real-world use.

This testing process encompassed all core functionalities of the platform, including user authentication, profile management, product browsing, chatbot communication, skin type prediction via face scanning, shopping cart interactions, and the end-to-end checkout process. Each function was tested independently and in combination with related components to ensure consistent performance under varying conditions.

Testers verified that all input fields accepted valid data and properly handled invalid or missing inputs through validation messages and alerts. Workflows were assessed to confirm that they followed the intended logic for example, ensuring that only registered users could add products to the cart or that the checkout process could not be completed without entering shipping and payment details.

Edge cases and boundary values were tested to identify any limitations or vulnerabilities in the system's logic. The results were documented meticulously, and any discrepancies between expected and actual outcomes were logged for further investigation and resolution by the development team.

Functional testing also ensured that user roles, such as admins and customers, had access only to their designated features and functionalities. By confirming that RadiantAura functioned correctly under all expected scenarios, this phase contributed significantly to the overall reliability and quality of the software product.

Table 6.3: Functional Testing

Test Case ID	Functionality	Test Case Description	Expected Result	Actual Result
FT-001	User Registration/Login	Register a new user	User registers and logs in successfully	Registration and login successful
FT-002	Skin Analysis + Chatbot	Capture an image, analyze skin, ask for product advice	Analysis and advice workflow functions seamlessly	Full workflow completed successfully
FT-003	Product Recommendation	Generate recommendations based on updated skin profile	Updated suggestions reflect profile changes	Recommendations updated accordingly

6.4 Integration Testing

Integration testing is a critical phase in the software testing life cycle that focuses on verifying the interactions between various modules and components within the RadiantAura system. This testing was carried out to ensure that individually tested units, such as authentication, product management, chatbot, face scanner, shopping cart, and checkout modules, function seamlessly when combined.

During this phase, multiple modules were integrated incrementally and tested as a group to identify interface-level defects, data inconsistencies, and logical errors in the flow of information between components. For instance, the integration between the user authentication module and the admin dashboard was tested to confirm that only authorized users could access administrative functionalities. Similarly, the link between the chatbot and the skincare recommendation engine was verified to ensure user input passed through the natural language processing layer and produced accurate advice.

The interaction between the face scanner and the backend AI model was also validated, focusing on how the image input was captured, sent to the server, processed, and returned with a suitable skin type and product recommendation. Another key integration involved testing the entire shopping journey from adding products to the cart, proceeding to checkout, entering user details, and generating order confirmations to ensure the data persisted correctly across modules and the user experience remained uninterrupted.

Overall, integration testing for RadiantAura ensured that combined components of the system worked together as intended, and data exchange between modules was accurate, secure, and in accordance with the application's workflow requirements.

Table 6:4: Integration testing

Test Case ID	Components Involved	Test Case Description	Expected Result	Actual Result
IT-001	Camera + Scanner	Live image feeds directly into analysis logic	Skin data extracted from real-time input	Integration successful
IT-002	Scanner + Recommendation	Analyzer output used to fetch product suggestions	Products matched with detected skin type	Products matched skin type well
IT-003	Chatbot + Analyzer	Ask for product type based on analysis	Chatbot uses skin type to guide product answer	Chatbot returned matched recommendations

6.5 System Testing

System testing was conducted to evaluate the complete, integrated system of RadiantAura and to ensure that all components worked seamlessly together in a real-world environment. This phase aimed to validate the end-to-end workflow of the application, examining how different modules interact and how well the system performs when executing actual business scenarios from start to finish.

The testing involved checking the flow of user activities from registration and login to browsing products, interacting with the chatbot, scanning skin type using the face scanner, adding products to the shopping cart, proceeding through checkout, and confirming orders. Each step was tested to confirm that the system maintained data integrity, proper navigation, and expected outputs throughout the entire process.

Both frontend and backend operations were evaluated to ensure that data submitted by the user was accurately processed, stored, and retrieved by the system. Special attention was given to

system responsiveness, error handling, session management, and proper redirection during user actions. All functionalities were tested in an environment that closely mimicked the production setup, ensuring that the system could handle real-time operations smoothly.

System testing also included checks for usability, performance under various conditions, and compatibility across different devices and browsers. By thoroughly validating the complete workflow of RadiantAura, this phase ensured that the system was fully functional, stable, and ready for deployment.

Table 6.5: System Testing

Test Case ID	System Components	Test Case Description	Expected Result	Actual Result
ST01	Full Real-Time Workflow	Start app → Search → Chat → Scan → Recommend	All features work in sequence	System responded correctly at each step
ST02	Permission Handling	Deny camera access initially	App prompts correctly and handles denial	Handled with user guidance
ST03	Lightweight Session Behavior	Use app for multiple scans in one session	Each scan is fresh and does not carry over	Session data was isolated and resettable

Chapter 7

7 Conclusion and Future Work

7.1 Conclusion

In conclusion, RadiantAura stands as a groundbreaking advancement in the beauty and skincare industry, offering an innovative approach to personalized skincare and cosmetic consultations through the power of artificial intelligence. By seamlessly integrating advanced machine learning algorithms, facial analysis techniques, and a responsive chatbot interface, RadiantAura delivers a truly intelligent and dynamic experience for users seeking tailored beauty guidance.

The system leverages facial recognition and image processing to accurately detect various skin types and conditions, enabling precise recommendations for skincare routines and cosmetic products. This automated skin analysis eliminates the guesswork traditionally associated with skincare decisions, empowering users to make informed choices based on scientific assessment rather than trial and error. The use of AI ensures that the recommendations evolve over time, adapting to user feedback and skin progress for continuous personalization.

RadiantAura's chatbot plays a pivotal role in making the platform interactive and user-friendly. By offering real-time conversations, the chatbot can interpret user concerns, suggest appropriate products, answer queries, and even guide users through the skincare regimen step by step. This enhances customer engagement, fosters trust, and replicates the experience of having a personal beauty consultant available 24/7.

Furthermore, the platform integrates a secure authentication system, ensuring that users' data and personal information are protected at all times. This instills confidence among users while interacting with the platform, especially when storing sensitive data like facial scans and skincare history.

From a technical standpoint, RadiantAura showcases robustness and reliability through well-structured backend architecture, modular components, and thorough testing practices such as unit, integration, functional, and system testing. These efforts ensure that each feature operates smoothly, efficiently, and with minimal errors.

RadiantAura's success lies not only in its technological sophistication but also in its commitment to enhancing user experience. By focusing on personalization, ease of use, and responsiveness, the platform addresses real-world needs in an intuitive and meaningful way. It redefines the concept of beauty consultation by bridging the gap between AI innovation and human-centric design.

Ultimately, RadiantAura sets a new benchmark for digital beauty advisory systems. It exemplifies how AI can be applied thoughtfully to solve real consumer problems, offering a vision for the future of personalized skincare and cosmetic care smarter, faster, and tailored for every individual.

7.2 Future Work

While the current version of RadiantAura is fully functional and provides a seamless and intelligent skincare shopping experience, there is significant potential for future expansion to enhance its effectiveness, accessibility, and market reach. The following enhancements have been identified to make RadiantAura more advanced, scalable, and competitive in the beauty and skincare domain:

7.2.1 Augmented Reality (AR) Integration

Future versions of RadiantAura could integrate AR-based virtual try-on features, allowing users to visualize how skincare or cosmetic products like moisturizers, foundations, or sunscreens will look and feel on their skin in real time using their phone cameras. This will significantly improve the personalization of product selection and reduce hesitation and product returns.

7.2.2 Advanced AI-Based Personalized Recommendations

By further enhancing machine learning models, RadiantAura can offer even more intelligent product suggestions based on a combination of facial scans, skin type, purchase history, seasonal needs, and skincare goals. This would help create highly personalized skincare journeys, improving user satisfaction and retention.

7.2.3 Community Reviews and Product Ratings

Implementing a reviews and ratings system will allow users to share their experiences with products. Verified customer reviews, skin-type-specific feedback, and before-after photos could help new users make informed decisions and foster a sense of trust and community within the platform.

7.2.4 Subscription Plans and Loyalty Programs

RadiantAura could introduce monthly or quarterly skincare subscription boxes tailored to users' skin conditions and goals. A reward-based loyalty program encouraging regular engagement, referrals, and feedback could also help in user retention and brand building.

7.2.5 Globalization Support (Languages and Currency)

To cater to a broader audience, future versions of RadiantAura should include multi-language interfaces and currency support. Localizing the UI, product labels, and chatbot conversations will make the app more accessible and appealing to users across different regions.

7.2.6 Social Media and Influencer Integration

Social sharing features that let users post their skincare results, product wishlists, or chatbot recommendations on platforms like Instagram, TikTok, or Facebook can organically boost visibility. Influencer partnerships and campaign integration could be facilitated directly within the app for better brand reach.

7.2.7 Advanced Admin Dashboard and Analytics

Enhancing the admin panel with dashboards that provide deep insights into customer behavior, trending products, sales metrics, and skin-type statistics can help administrators make informed decisions. Features like inventory forecasting, campaign tracking, and sales heatmaps could also be integrated.

7.2.8 Accessibility Improvements

To promote inclusivity, RadiantAura could be enhanced with accessibility features such as voice-enabled search, chatbot voice interaction, screen reader compatibility, high-contrast themes, and simplified UI for elderly or differently-abled users.

Chapter 8

References

- [1] J. Duckett, *HTML & CSS: design and build websites*. Indianapolis, Indiana: John Wiley & Sons Inc, 2014.
- [2] “Schwaber, K. and Beedle, M. (2002) Agile Software Development with Scrum. Prentice-Hall, Upper Saddle River. - References - Scientific Research Publishing.” Accessed: May 16, 2025. [Online]. Available: <https://www.scirp.org/reference/referencespapers?referenceid=1150808>
- [3] “MySQL.” Accessed: May 16, 2025. [Online]. Available: <https://www.mysql.com/>
- [4] “Installation - Laravel 10.x - The PHP Framework For Web Artisans.” Accessed: May 16, 2025. [Online]. Available: <https://laravel.com/>
- [5] “BEAUTY ADVISOR,” BEAUTY ADVISOR. Accessed: May 16, 2025. [Online]. Available: <https://beautyadvisor.com.ua>
- [6] “Beauty Buddy |.” Accessed: May 16, 2025. [Online]. Available: <https://thebeautybuddy.com/>
- [7] “Dermalogica® Skin Care | Official Site.” Accessed: May 16, 2025. [Online]. Available: <https://www.dermalogica.com/>
- [8] “AI & AR Solutions for Skin, Beauty, Fashion | YouCam App Provider.” Accessed: May 16, 2025. [Online]. Available: <https://www.perfectcorp.com/business>
- [9] “Recommended Skincare Brand for Sensitive Skin | Cetaphil US.” Accessed: May 16, 2025. [Online]. Available: <https://www.cetaphil.com/us/>
- [10] “perfectcorp.com/business.” Accessed: May 16, 2025. [Online]. Available: <https://www.perfectcorp.com/business>
- [11] “Visual Studio Code - Code Editing. Redefined.” Accessed: May 16, 2025. [Online]. Available: <https://code.visualstudio.com/>
- [12] “Download Android Studio & App Tools - Android Developers.” Accessed: May 16, 2025. [Online]. Available: https://developer.android.com/studio?gad_source=1&gad_campaignid=22548400775&gbraid=0AAAAAC-IOZILMbwYcOamyQAJsNqVwOTG8&gclid=Cj0KCQjwoZbBBhDCARIsAOqMEZW1rbJydII4NYYVHJ0Kf5q1A0XLLTt6yG7a84Fdj4YIqHvh5qfQFHAApXREALw_wcB&gclsrc=aw.ds

[13] “Buy a domain name - Register cheap domain names from \$0.99 - Namecheap.”

Accessed: May 16, 2025. [Online]. Available:

[https://www.namecheap.com/?gad_source=1&gad_campaignid=11301910042&gbraid=0AA
AADzFe22eFF1K3HTiQ3CSNjoPShrTY&gclid=Cj0KCQjwoZbBBhDCARIsAOqMEZUI
8jLx1qT4uMUMOOdLPBzMtUxB5qZzZ52DvvWbnZsHGwdHPbh63wwaAoOrEALw_wc
B](https://www.namecheap.com/?gad_source=1&gad_campaignid=11301910042&gbraid=0AA
AADzFe22eFF1K3HTiQ3CSNjoPShrTY&gclid=Cj0KCQjwoZbBBhDCARIsAOqMEZUI
8jLx1qT4uMUMOOdLPBzMtUxB5qZzZ52DvvWbnZsHGwdHPbh63wwaAoOrEALw_wc
B)

[14] “Introduction · Bootstrap v5.0.” Accessed: May 16, 2025. [Online]. Available:

<https://getbootstrap.com/docs/5.0/getting-started/introduction/>

[15] “Home - Canva.” Accessed: May 16, 2025. [Online]. Available:

<https://www.canva.com/>

[16] “Home – Figma.” Accessed: May 16, 2025. [Online]. Available:

[https://www.figma.com/files/team/1193905507311740570/recents-and-
sharing?fuid=1193905499960232676](https://www.figma.com/files/team/1193905507311740570/recents-and-
sharing?fuid=1193905499960232676)

[17] “PHP: Hypertext Preprocessor.” Accessed: May 16, 2025. [Online]. Available:

<https://www.php.net/index.php>

[18] “Composer.” Accessed: May 16, 2025. [Online]. Available: <https://getcomposer.org/>

[19] “XAMPP Installers and Downloads for Apache Friends.” Accessed: May 16, 2025.

[Online]. Available: <https://www.apachefriends.org/>

[20] “Android Apps on Google Play.” Accessed: May 16, 2025. [Online]. Available:

<https://play.google.com/store/games?hl=en>